

Differential Fuzzing Framework For RRDP Protocol

Master thesis in the department of Computer Science by SWARNALI SARKAR

Date of submission: March 19, 2026

1. Review: Prof. Dr. Michael Waidner
2. Review: Prof. Dr. Björn Scheuermann
Darmstadt



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Computer Science
Department

Information Technology
Security (SIT)

Erklärung zur Abschlussarbeit gemäß § 22 Abs. 7 APB TU Darmstadt

Hiermit erkläre ich, Swarnali Sarkar, dass ich die vorliegende Arbeit gemäß § 22 Abs. 7 APB TU Darmstadt selbstständig, ohne Hilfe Dritter und nur mit den angegebenen Quellen und Hilfsmitteln angefertigt habe. Ich habe mit Ausnahme der zitierten Literatur und anderer in der Arbeit genannter Quellen keine fremden Hilfsmittel benutzt. Die von mir bei der Anfertigung dieser wissenschaftlichen Arbeit wörtlich oder inhaltlich benutzte Literatur und alle anderen Quellen habe ich im Text deutlich gekennzeichnet und gesondert aufgeführt. Dies gilt auch für Quellen oder Hilfsmittel aus dem Internet.

Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Mir ist bekannt, dass im Falle eines Plagiats (§38 Abs.2 APB) ein Täuschungsversuch vorliegt, der dazu führt, dass die Arbeit mit 5,0 bewertet und damit ein Prüfungsversuch verbraucht wird. Abschlussarbeiten dürfen nur einmal wiederholt werden.

Bei einer Thesis des Fachbereichs Architektur entspricht die eingereichte elektronische Fassung dem vorgestellten Modell und den vorgelegten Plänen.

English translation for information purposes only:

Thesis Statement pursuant to § 22 paragraph 7 of APB TU Darmstadt

I herewith formally declare that I, Swarnali Sarkar, have written the submitted thesis independently pursuant to § 22 paragraph 7 of APB TU Darmstadt without any outside support and using only the quoted literature and other sources. I did not use any outside support except for the quoted literature and other sources mentioned in the paper. I have clearly marked and separately listed in the text the literature used literally or in terms of content and all other sources I used for the preparation of this academic work. This also applies to sources or aids from the Internet.

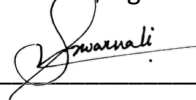
This thesis has not been handed in or published before in the same or similar form.

I am aware, that in case of an attempt at deception based on plagiarism (§38 Abs. 2 APB), the thesis would be graded with 5,0 and counted as one failed examination attempt. The thesis may only be repeated once.

For a thesis of the Department of Architecture, the submitted electronic version corresponds to the presented model and the submitted architectural plans.

Datum / Date:
19.03.2026

Unterschrift/Signature:



Contents

1	Introduction	11
1.1	Motivation	14
1.2	Problem Statement	15
1.2.1	A Possible Approach Overview	17
1.3	Contributions	19
2	Related work	21
2.1	Introduction	21
2.1.1	Fuzzing at the Protocol Layer	22
2.1.2	Understanding of Differential Fuzzer	22
2.1.3	Structure Aware Fuzzing	23
2.1.4	Security Risk in XML Document	23
2.1.5	Previous Work on RPKI and RRDP Security	23
3	Technical Background	25
3.1	Overview	25
3.2	Inter-Domain Routing System	25
3.3	Resource Public Key Infrastructure (RPKI) Architecture	26
3.3.1	What is RPKI and how it works in the Internet	26
3.3.2	Certificates and Signature:	27
3.3.3	Route Origin Validation (ROV)	29
3.3.4	RPKI Validators	29
3.3.5	Routers	30
3.4	Understanding of RPKI Repository Delta Protocol (RRDP) working flow . .	31
3.4.1	What is RRDP?	31
3.4.2	How a validator discovers RRDP	31
3.4.3	Definition of RRDP files	32
3.5	Working Model of RRDP protocol	34
3.5.1	Polling and Tracking	36



3.5.2	Synchronization Path Decision	36
3.5.3	Integrity and Consistency Validation	36
3.6	Challenges in RRDP implementation	37
4	Methodology & Approach - Implementation of RRDP Differential Fuzzer Framework	38
4.1	The Fuzzing Workflow	39
4.2	Approach 1 - Structure-Aware XML Mutation	40
4.2.1	Overview	40
4.2.2	Mutation Pipeline	41
4.2.3	Recursive Tree Traversal	44
4.2.4	Protocol-Guided Mutation Decisions	44
4.3	Approach 2 - Overview of Orchestrator Workflow	46
4.4	The Orchestrator Architecture	48
4.4.1	Baseline Validation	48
4.4.2	Mutation Strategy	50
4.4.3	Validator Validation Process and Result Unification	51
4.4.4	Differential Oracle	52
4.4.5	Case Reproducibility	55
5	Evaluation of Differential Fuzzer	56
5.1	Introduction	56
5.2	Experiment Overview	56
5.3	Analysis 1 - Overall Evaluation of Fuzzer	58
5.4	Analysis 2 - Inconsistent Behavior Across the Validators	59
5.4.1	Understanding of Parser Differential Behavior for Routinator	62
5.4.2	RFC 8182 XML Handling	63
5.4.3	Analysis of Mutated XML Files Causing Disagreement	63
5.4.4	Unknown Attributes	64
5.4.5	Examples of unknown Attribute Mutations in XML Files	65
5.4.6	False Positive Analysis	65
5.4.7	Test cases Repeatability	66
5.5	Analysis 3 - Raw Differentials	69
5.5.1	Overview of Raw Differentials	69
5.5.2	Raw Differential Logs for a Structural RRDP Mutation	70
5.5.3	Analysis of Dataset	71
5.6	Contextualization of Results	74

6	Limitations	75
6.1	Future Improvement	78
6.2	Future Work	79
7	Conclusion	80
8	Acknowledgment	81

List of Figures

1.1	End-to-end RPKI validation pipeline from trust anchors through repository synchronization, validator processing, VRP generation, and BGP route origin validation.	13
3.1	RPKI trust anchor hierarchy showing the five Regional Internet Registries (RIRs) and their certificate-based relationship to the IANA root trust anchor.	28
3.2	Structure of RRDP XML files and their relationships. The Notification.xml file references both the Snapshot.xml file and the Delta.xml files via URIs and a SHA-256 hash value.	33
3.3	RRDP XML files synchronization workflow from the point of view of a Relying Party. It describes how to handle sessions, choose between delta and snapshot, and check hashes.	35
4.1	Overview of a Differential Fuzzer Work model, showing baseline verification, mutation loop, multi-validator execution, and differential oracle filtration.	49
4.2	Differential oracle workflow for classifying validator disagreements.	53

List of Tables

5.1	Overall Fuzzing Summary	58
5.2	Classification of VRP output patterns across validators. R = Routinator, C = rpki-client, F = Fort. Values indicate VRP count (0 = rejected, 100 = accepted). Status indicates whether the pattern qualifies as a clean differential after replay and metadata consistency checks.	61
5.3	XML event handling in rpki-rs and its relationship to well-formedness and the RELAX NG schema.	62
5.4	Appearance of Targeted Mutation corpus.	64
5.5	Types and repetition of unknown attribute that appeared in xml files . . .	64
5.6	Representative unknown-attribute mutation cases.	65
5.7	Verification of snapshot hash consistency for the representative case iter_00014, this case shows inconsistencies across validator.	66
5.8	Repeatability of replay for oracle case iter_00014	67
5.9	Verification of replay metadata consistency for oracle case iter_00014. .	69
5.10	Archived artifact coverage by validator and size of the common three validators comparison subset.	72
5.11	Distribution of mutation patterns identified across all mutated snapshot.xml files, grouped by coarse family analysis. Percentages are relative to the total number of mutated snapshots (4296).	73



List of Algorithms

1	Protocol-guided RRDP XML mutation	45
2	Differential fuzzing orchestration	47
3	Differential oracle collection and clean reporting	54

List of Abbreviations

RPKI	Resource Public Key Infrastructure
RRDP	RPKI Repository Delta Protocol
ROA	Route Origin Authorization
RP	Relying Party
TAL	Trust Anchor Locator
VRP	Validated ROA Payload
CA	Certificate Authority
CRL	Certificate Revocation List
XML	Extensible Markup Language
AST	Abstract Syntax Tree
URI	Uniform Resource Identifier
INR	Internet Number Resources
RIRs	Regional Internet Registries
SIA	Subject Information Access
OID	Object Identifier
RFC	Request for Comments
IETF	Internet Engineering Task Force

Abstract

The RPKI Repository Delta Protocol (RRDP) allows RPKI validators to fetch and synchronize repository data needed for Route Origin Validation (ROV). In practice, multiple independent validators process the same RRDP repository, but they do not always handle it in the same way. These differences in behavior can affect the reliability of routing security validation. Since RRDP uses XML-based files that include `notification.xml`, `snapshot.xml`, and `delta.xml` files, validator behavior depends on both protocol logic, cryptographic verification, and XML parsing decisions [1]. Despite this, systematic testing of RRDP processing behavior across different implementations remains limited.

This research introduces the RRDP-Diffuzz framework, a differential fuzzing system for testing RRDP protocol-based RPKI validators in a controlled environment. The framework applies structure-aware XML mutation to RRDP XML files, repeatedly runs the RPKI validators on the same crafted RRDP XML files, and compares their output using a differential oracle filtration. This proposed design enables the automated generation of structured test inputs that are structurally valid and can be processed by validators to expose behavioral disagreement across implementations.

This study aims to evaluate the robustness of RRDP validators by facilitating the systematic generation and comparison of mutated RRDP repository states. It also serves as a foundation for future extensions, such as delta-chain fuzzing, smart oracle filtering, and root cause analysis.

1 Introduction

BGP, the Border Gateway Protocol, serves as the Internet's inter-domain routing protocol that enables autonomous systems across the Internet to exchange routing information. The Internet routing system depends on the BGP announcement system that decides how the traffic should flow across multiple administrative domains. BGP is essential for maintaining connectivity worldwide because of its distribution and trust-based architecture, but it has also been identified as a long-standing source of security concern.

Initially, BGP was not built with strong security mechanisms to check whether a prefix belongs to a legitimate source. It is all about trust-based, which means any autonomous system can announce unauthenticated routes over the resources. As a result, the Internet is vulnerable to attacks like route hijacking, route leaking, and misconfiguration. Whenever an inaccurate routing announcement appears across the network, they spread quickly and causes extensive outages, emphasizing the critical need for dependable mechanisms to validate routing information.

There are many incidents of routing security, one of the notable incidents happened in 2018, where the attackers had announced more specific BGP routes for an IP address ranges linked to a popular cloud service provider. As a result, the cryptocurrency traffic was temporarily diverted to an attacker-controlled fake website, which led to theft of cryptocurrency worth approximately \$150,000 USD [2].

This incident describes how a BGP route hijacking can be used in traffic interception and credential theft. Furthermore, this emphasizes that security failures in the route can create an immediate financial crisis and affect services that are relying on the cloud platform.

To mitigate these limitations, the Resource Public Key Infrastructure (RPKI) was introduced as a cryptographic system that allows network operators to verify the legitimacy of route origin announcements. To obtain trustworthy routing information for the BGP decision-making process, RPKI validators fetch repository data, validate RPKI objects, such as

cryptographic objects, Route Origin Authorizations (ROAs), certificates, Trust Anchor Locators (TALs), and produce validated routing information. Therefore, the accuracy and reliability of RPKI validators directly affect security and stability.

Certificate Authorities (CAs) publish RPKI repository data on repository servers, which validators can download through various synchronization mechanisms, such as rsync, and most importantly via the RPKI Repository Delta Protocol (RRDP) [3]. The RRDP allows validators to update the repository's incremental state through HTTPS, downloading only the data that has changed, rather than the entire repository, each time. This method makes it more efficient and popular due to its scalability and deployment.

Figure 1.1 demonstrates the complete structure of the RPKI validation workflow. This shows a validation pipeline from the hierarchical distribution of resources by IANA and the RIRs, through the repository synchronization via RRDP/rsync protocol, to the generation of VRP lists that classify routes as valid or invalid.

This workflow relies on multiple validators that are built with individual designs. They all process the same repository data. If any of these validators behave in a way that affects the routing decisions, then it leads to a concern regarding how reliably these validators handle the same input in an RRDP system.

RPKI Validation Overview

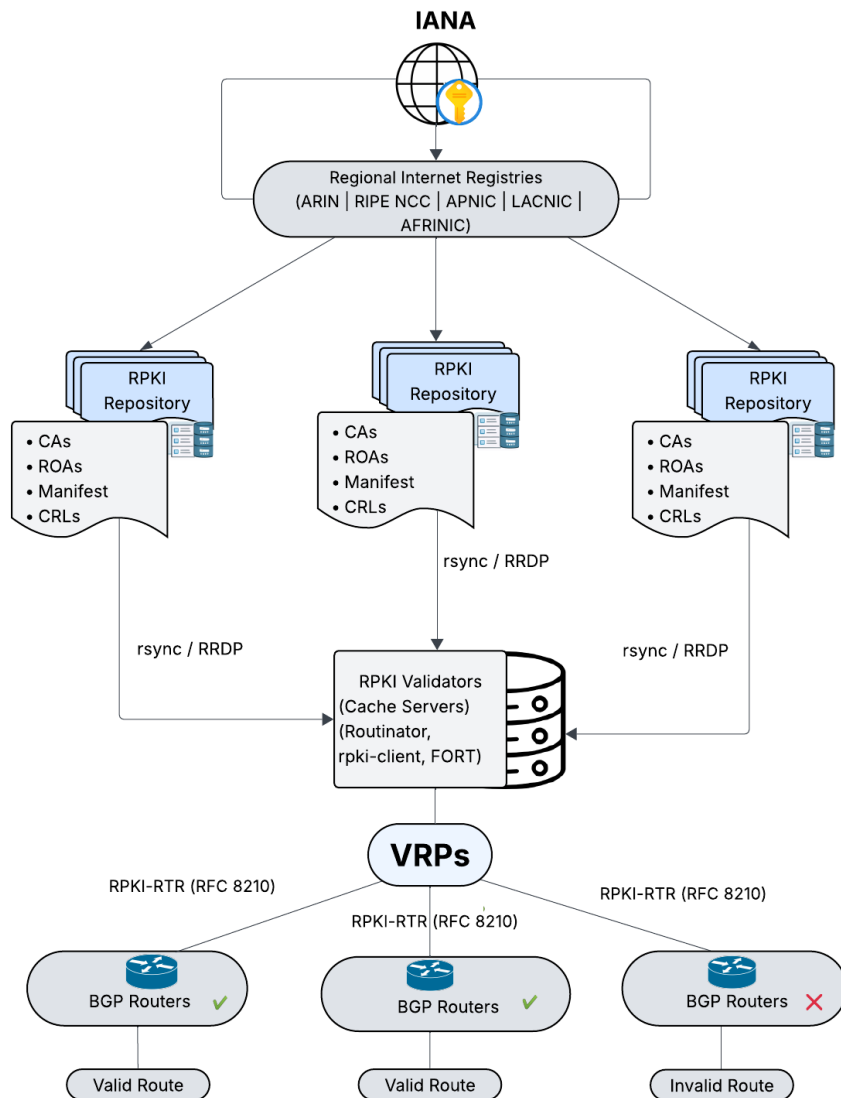


Figure 1.1: End-to-end RPKI validation pipeline from trust anchors through repository synchronization, validator processing, VRP generation, and BGP route origin validation.

1.1 Motivation

The validators have become essential components of routing security as more networks adopt RPKI for Route Origin Validation (ROA) [4]. The validators act as a bridge between the operational routing decisions that the network makes and the cryptographic repository data, and the outcome directly affects the network decision on what route to trust for the announcements. This defines that validators work correctly for RPKI to be reliable.

Nowadays, all the RPKI validators are built with the RRDP protocol logic [3]. Even though the validators follow the same protocol rules, they don't always behave in the same way during the validation [1]. The reason for these divergent behaviors during the time of the validation process is due to their architectural design and implementation. The validation sequence and error handling process of validators are different from each other, and they also interpret unclear parts of the specification in their own ways.

Imagine a scenario where an RPKI repository unintentionally publishes an RRDP update, which is semantically unstable but syntactically valid, for instance, an improper state of transition between subsequent repository snapshots and deltas. This kind of circumstance arises from an operational error, an incorrect update of the repository, or a bug in the repository management system, which may reduce the trust policy in the system.

In these situations, RPKI validators may respond differently to the same repository state. Where one validator may accept an update and proceed with the validation using insufficient data, and another may reject the update and revert to a previous state. This divergence may result in conflicting routing decisions in different networks from the same RPKI repository.

These divergences can create serious issues in security infrastructure. Recent measurements confirm that 56% of globally deployed RPKI validators are affected by at least one known vulnerability [5]. Prior research has demonstrated that attackers can exploit implementation weaknesses to disable validators entirely, further threatening the stability of routing security [6]. This makes the validation system less reliable than network operators expect.

These behaviors are a point of concern as they are not identified during normal operations. They are normally observed when there are unexpected inputs in the repository or in the case of malicious attacks. However, it is not possible to identify such behaviors using traditional testing methods such as manual test cases and single-implementation fuzzing, as these methods do not compare different validator behaviors. Security is a significant

aspect in BGP systems, and such unstable behaviors may reduce trust in the validation output.

According to the RPKI Repository Delta Protocol (RFC 8182) [3], independently built RPKI validators often show divergent behavior when they process the same inputs. However, this outlier consistency can cover the underlying divergences that can emerge only under unusual circumstances, such as malformed data, unexpected inputs, or problematic repository states. These situations may result from misconfigurations, mistakes, intentional attacks, or software bugs.

From an operational standpoint, the inconsistent behavior of the validator weakens trust in RPKI as a routing security tool. When different networks reach debatable conclusions about the same routing information, coordination becomes difficult, and trust in validation results falls. This requires systematic testing methods that can go beyond standard operational scenarios to actively explore edge cases in protocol behavior [1].

These observations have pointed out an issue with the behavioral variations among RPKI validators during the validation process of RRDP repositories. Therefore, these issues are described in the following section.

1.2 Problem Statement

Despite having standardized specifications for the RPKI Repository Delta Protocol (RFC 8182) [3] and related RPKI protocols, independently built RPKI validators often show divergent behavior when processing the same RRDP repository as input. Differences in how protocols are interpreted, how internal state is managed, and how errors are handled can lead to divergent behaviors. These differences may lead to variations in the output generation decisions of the RPKI validator's validation process, inconsistent error reporting, or differences in the generated Validated ROA Payloads (VRPs).

Detecting such inconsistencies in the system is challenging. The traditional testing techniques mostly rely on manually generated test cases, which show expected or normal protocol behavior. While these tests can check for basic correctness, they fall short in exploring the many possible repository states and edge cases that might occur in real situations. Additionally, testing a single validator in isolation does not provide any insight into how it compares with other implementations.

The lack of a standard definition of correctness makes testing RRDP validators even more complicated. In many cases, the protocol specification can be open to various interpretations or might not clearly define how things should behave in certain cases. It is not always easy to determine whether a validator's behavior is completely correct or incorrect when looked at by itself. However, noticeable variations in validators on handling the identical inputs strongly suggest that at least one implementation might be showing unintended or undefined behavior.

Ignoring inconsistencies in RPKI validators is a major risk because they play a vital role in security. Different validation results can result in disparate policies across networks, erode the RPKI trust, and could enable the door for exploitation through crafted repository states. A testing strategy that can reliably and efficiently identify the behavioral variations among the validators without relying on a rigid notion of correctness is required to address this issue [7]

The following research questions and the investigations serve as the base of this thesis:

- Whether a differential fuzzing framework can detect behavioral inconsistencies systematically across multiple individually designed RPKI validators based on the RRDP protocol when parsing the same mutated repository?
- Following the first question, how can a standalone structure-aware mutation engine produce syntactically valid, diverse mutated XML files?
- Whether a differential Oracle can distinguish the true divergence or inconsistencies found in validators caused by a mutated repository or any other circumstances during the validation process?
- If any differential behavior is observed, then what is the cause of that inconsistency?

To overcome all these issues, we have approached a methodology that will improve the reliability of the validation process and reduce the individual manual testing of RPKI validators. The next section discusses a possible solution.

1.2.1 A Possible Approach Overview

The challenges of the validation process can be solved by implementing a differential fuzzing system for testing RPKI validators. Differential fuzzing compares the behavior of multiple independent implementations by running them on the same inputs. This methodology observes the differences in their visible behavior. Instead of trying to determine absolute correctness, this method looks for inconsistencies between validators as signs of possible bugs, unclear specification behavior, or missing validation logic.

The core foundation idea of differential Fuzzing is straightforward. It offers an effective detection method for finding divergence behavior across multiple validators. The system runs with identical input in multiple implementations and compares how exactly they behave. There is no requirement for predefined, accurate answers. Whenever the validators disagree on something or provide different results for different validators, it means there is a bug or an event of malicious data. It might mean improper specification or a lack of validation logic. This approach works very well for standardized protocols, like RRDP, that have multiple independent implementations.

This proposed method is designed for RPKI validators that are based on the RRDP protocol. First, there are multiple validators (such as Routinator, Fort, rpki-client, OctoRPKI, and others), and they have separate implementation structures. Differential testing is made with possible implementation diversity. Second, it is simple to supply identical inputs to multiple RPKI validator implementations as they are made to process standardised input formats (RRDP XML files as defined in RFC 8182). Third, validators generate observable outputs that can be compared programmatically, including logs, error messages, sets of Validated ROA Payloads (VRPs), and success or failure status of the validation process. Finally, since no single validator can be presumed to be accurate due to the lack of an authoritative reference implementation, differential analysis is the most suitable methodology for identifying inconsistencies.

RRDP-Diff fuzz Framework

This framework is designed with three main core functions:

1. This system is designed to automatically generate a structurally valid and invalid RRDP repository within a controlled environment. The mutator crate applies a modification to valid rrdp XML files. The modifications it applies are structural mutations like child

reordering, element insertion/deletion/duplication, notification-level URI and hash perturbations, and XML attacks (comment injection, CDATA wrapping, namespace prefix changes, whitespace manipulation [8] when preserving the original cryptography integrity.

2. Running the validators against the same input in an isolated, controlled environment.
3. Observation and classification of behavioral divergences through systematic comparison of validator outputs.

Basically, this fuzzing system treats all the validators as "black boxes" and observe their behaviors that are visible to the outside world, without requiring access to internal instrumentation, source code, or debugging data. The framework records logs for each validator, whether it completed validation successfully for each test case, along with the VRPs, error or warning messages, exit codes, and status indicators that each validator produced. By comparing these generated outputs between validators, the system identifies the instances where the builds differ; for example, one validator rejects an RRDP update while another accepts it, or when validators generate distinct sets of VRPs from the same repository state.

The framework mainly focuses on exploring deeper validation logic rather than merely producing a simple parsing error. To achieve the goal, the fuzzing framework implementation approached with structural XML element mutation strategies; the structure-aware mutation, which maintains the structure validity while applying mutation to a specific element of the XML files. For instance, the system generates well-formed XML files with a modified serial number, session_id, hash byte, & URI rather than producing random XML that validators reject at the early stage of validation. This test approach is integrated to examine whether the validation can reach a RRDP-specific logic where built-in differences are more likely to be manifest.

Test Generation With Mutation

The framework generates test cases by carefully modifying the RRDP repository. It starts with producing a valid baseline that is followed by RFC 8182 [3] and it includes a notification file, a snapshot file, and delta files in its repo state. Then the mutation crate is applied with specific edits in the protocol fields and to the XML structure. Such as:

- **Protocol-aware mutations:-** It applies document type structural edits. For example, the notification file may mutate the hash attribute by flipping a bit or a hex digit, the URI attributes with fake URLs on snapshot file, or the delta elements. As mentioned

earlier in the RRDP-Diffuzz framework, the crate never changes or modifies the publish or withdraw element, as the cryptographic integrity is linked to the RPKI manifest.

- **Structure-aware mutations:-** Applies controlled modification in the targeted element of the existing XML structure. The modification occurs in five stages: In stage 1 and 2, it's the Reordering of child elements to check their order dependency, and inserting bogus elements, deleting children, and duplicating them are all tree-based structural mutations. In stage 3 and 4 XML Signature Wrapping - basically, this stage applies mutation into crypto validation by flipping a bit of the hash and URI replacement [9]. Stage 5 applies bypass web application method by adding name space, whitespace, and CDATA [8]. When applying mutation on the XML files, all the stage maintains the valid structure of the XML tree so that the validators can explore deeper logic without failing at the early stage of validation.

By combining both test cases, the framework explores many different RRDP repository states, including rare edge cases and malformed inputs that normally would not appear, but could reveal weaknesses or differences between validators.

Deterministic and Reproducible Testing

The RRDP-Diffuzz testing methodology is implemented to provide a robust result for RPKI validation. The system makes sure that the test cases are deterministic and reproducible across multiple runs by using fixed random seeds. Each test case has a unique ID, and the full repository state (notification, snapshot, and delta files) enables reproducing any divergence by using only the execution logs. Generally, it targets subtle cache instability logic bugs and validation errors in Relying Party (RPs).

All the validators are run in isolated environments to maintain the test integrity and to avoid interference with each other. Each validator has its own cache folder (cache, config, TALs) to ensure that the results are independent of all previous runs. Additionally, this framework emulates a network using a local nginx server that acts as a real server and serves all controlled repositories to the validators during the test.

1.3 Contributions

The contributions of this work are for designing the RRDP-Diffuzz fuzzing tool:

-
- **Isolated Lab:** A local RRDP lab set up that isolates the RPKI validator to serve from the locally served "rrdp" repository instead of fetching from the external server. This lab was built with an "**nginx**" web server, but it was configured with an nginx **alias** to serve the current repo to the validators.
 - **Valid Repository Generation with CURE:** Added `cure_pp` and `cure_as1` tools into a local lab setup to generate a fully valid RPKI repository or a publish point, which serves the original seed input for differential fuzzing. This seed baseline is then served via `nginx` and reused in all executions to ensure that differences come from the mutations, not from an invalid starting repository.
 - **Structure-aware XML mutation crate:** Basically, this mutation engine is built to generate many RRDP repositories, which have variants with protocol-aware and structure-aware changes, so the validation does not fail at the early stage or at the "front-door" of the validator before driven into a deeper logic.
 - **Implementing Differential Fuzzer:** Designed to test multiple RPKI validators in parallel without interference. The idea is to discover behavioral differences or how they react when fetching both valid and invalid inputs from the targeted repo.
 - **Study of divergence behavior:** After running multiple validators for at least 50 – 100 iterations, it can explore the differences in serial handling, hash checking, session handling, and error recovery.
 - **Validators agreement rate:** It shows the metrics of RRDP-DiffFuzz on how consistent multiple RPKI validators are by giving hard numbers on agree or disagree when they run the same repository. This shows how consistent the RPKI ecosystem is.
 - **Release of fuzzing Framework:** It releases RRDP-DiffFuzz as an open-source framework so developers, operators, and researchers can test validators, reproduce divergences, and improve implementations and specifications.

Overall, this work executes many RRDP input variations and compares the validator outputs, and shows where and why they differ. In the next Chapter 2 this thesis discusses prior work on RPKI security, protocol fuzzing, differential testing, and XML security risks relevant to RRDP.

2 Related work

2.1 Introduction

This section describes the prior and existing work that is relevant to the RPKI security ecosystem and to the research context of the differential fuzzing system for finding behavioral differences among RRDP-based RPKI validators. The main purpose of this section is to explain the foundations of the approach applied in this research after examining the related previous studies in RPKI security, protocol fuzzing, differential testing, and XML security risks at the RRDP layer.

The existing research on RPKI security, including some known attacks and vulnerabilities and how validators can behave inconsistently when processing unusual or malformed repository inputs, emphasizes why testing RPKI validators is important for Internet routing security.

Secondly, it covers the fuzzing techniques developed for protocols in earlier research and shows how fuzzing is useful for exploring unexpected states in software when dealing with crafted data. Third, it reviews previous work on XML security risks at the RRDP protocol layer.

After reviewing all the earlier work, it was found that systematic differential analysis across RRDP-focused validators has not been sufficiently addressed in prior research. Therefore, this research is motivated to develop a differential fuzzer called RRDP-Diffuzz for RRDP XML files across multiple RPKI validators. In addition, this RRDP-Diffuzz framework is built on top of the existing CURE tool for RPKI validation.

2.1.1 Fuzzing at the Protocol Layer

Generally, Fuzzing is an automated testing tool that finds hidden bugs, crashes, or other unusual behavior in RPs or any software by injecting randomly generated or malformed inputs [10]. It has been widely used to find bugs in many software. When we talk about network protocols, fuzzing becomes valuable as protocols have strict input formats and complex systems, which makes it hard to manually create all the possible edge cases that can happen in real-world use.

In general, protocol implementations are stateful, which means their behavior for any given input depends on the repository state they observed in previous validation cycles. This include order of messages, the current session state, and the protocol logic that can affect validators on how they behave. As a result, protocol fuzzing needs to test the message sequences instead of just one individual input. Fuzzing for network services and protocol software can get better by replaying valid protocol traces, changing protocol messages, and using feedback that knows the current state, according to earlier research [11], [12].

Additionally, there are many types of protocol fuzzing, one is generation-based fuzzing, another one is mutation-based fuzzing, and the last one is feedback-guided fuzzing. In Feedback-guided, the fuzzer uses the information it gets when running, such as code coverage and current state, to dig deeper into how things behave. AFL++ has demonstrated the effectiveness of combining mutation-based generation with coverage feedback to discover vulnerabilities across diverse software targets [13]. For generation-based fuzzing, it creates inputs based on the protocol model, and in Mutation-based fuzzing, the fuzzer modifies the valid inputs that already exist. As the RRDP handling depends on both the XML structure and how the repository behaves at the protocol level, [7] protocol fuzzing is useful for RRDP.

2.1.2 Understanding of Differential Fuzzer

Differential fuzzer is an automated testing tool that runs multiple independent software implementations on the same input and compares their outputs to identify behavioral divergences [10]. It is a method that helps to find inconsistencies, bugs, or any other issues by giving a mutated input to multiple software programs. During the validation process, if a meaningful disagreement is found, additional filtering or oracle-based triage is needed to separate genuine differentials from confused cases because not all disagreements correspond to actual bugs [14].

2.1.3 Structure Aware Fuzzing

Standard fuzzers work well for text or binary formats, but they struggle with complex inputs such as XML, JavaScript, or protocol messages that have strict rules. Even a single random bit change usually causes the parser to reject the input immediately before any interesting logic can run. Structure-aware fuzzing techniques address this complexity by keeping the input valid enough to be processed by the application instead of being rejected right away due to minor errors. Earlier studies have shown that techniques such as tree trimming, subtree mutation, and grammar-based mutation generation can greatly improve the effectiveness of fuzzing for structured formats. They help achieve better coverage and find more bugs [15], [16].

2.1.4 Security Risk in XML Document

Due to its role in data interchange, Extensible Markup Language (XML) is flexible but also introduces security and robustness risks. Official XML security guidance warns about risks such as malformed documents, external entity processing, parser resource exhaustion, and the handling of malformed XML input. For example, XML injection is a type of attack that exploits vulnerabilities in XML parsers and processors [8], [17].

For systems like the RRDP protocol that rely on XML for transport and data synchronization, problems can occur when handling malformed XML inputs. Even if an XML document is well-formed but contains unexpected attributes, unusual namespaces, or edge-case structural arrangements, it can still cause inconsistencies in parser behavior or error handling. Parser robustness is not only about validating the input; it is also related to protocol operational security. This makes previous work on XML security relevant for testing RRDP validators and for understanding parser hardening [3], [18], [19], [20].

2.1.5 Previous Work on RPKI and RRDP Security

Earlier studies on RPKI security showed that any weakness in the validation system can directly affect routing security. Studies on RPKI validation showed that the presence of under-specified or vague rules in the RFCs can lead validators to behave differently [4]. Further work reported that many vulnerabilities were found in validator implementations due to insufficient error handling [1]. Other major studies also stated that flaws in validator logic are prone to causing RP crashes, parsing errors, attacks, and route poisoning.

In addition, the CURE tool is introduced by developing an automated test for the RPKI security system. This system was developed to explore security flaws, bugs, and RFC compliance issues in RPKI validators (RP). RFC8182 standardized RRDP protocol to help synchronize RPKI repositories. Later RFC9674 made the RRDP model more secure by adding RRDP clients and servers policy.

Compared to CURE, this thesis has a reduced testing scope, narrowed from the RPKI object validation pipeline to the RRDP XML transport layer. While CURE relies on black-box, differential, and stateful fuzzing across the entire structure of RPKI repositories, it demonstrates the complexity of RPKI testing because it is a stateful system, and there is no single, strict, canonical behavior as described in the RFCs. This thesis, on the other hand, has a narrower testing scope, focusing on validator behavior for mutated RRDP XML structures, such as "notification.xml" and "snapshot.xml." This does not replace CURE but rather adds another dimension of testing, specifically the disagreement in the parser and RRDP layer, which is not directly tested in the RPKI object validation pipeline

All these studies show that testing RPKI validators is important for keeping the routing system secure. CURE has already built a differential and stateful fuzzer to test RPKI Relying Parties (RPs), especially targeting valid RPKI repository objects such as root CA certificates, Trust Anchor Locators (TALs), manifests, and Certificate Revocation Lists (CRLs). Also, CURE acts as a black-box differential fuzzer for comparing outputs such as validated ROA payloads or VRP lists. A recent study on RPKI security looked at all the weaknesses in the RPKI system [5]. It checked for attacks on RPKI, its infrastructure and how it is implemented. The study found that attacks that make RPKI unavailable are the most prominent threat. There have been at least 53 RPKI vulnerabilities reported in recent years. However, studies on RRDP validator behavior at the RRDP XML file layer remain underexplored in previous work. This is the point where this research work examines whether RPKI validators work correctly when they parse unexpected but well-formed XML files, and whether the built oracle-based differential fuzzer can identify these inconsistent behaviors in RPKI validators.

3 Technical Background

3.1 Overview

This chapter introduces the technical foundation required to design and implement Differential Fuzzer (RRDP-Diffuzz). It summarizes the Resource Public Key Infrastructure (RPKI) mechanism, the explanation of the inter-domain routing, including how RPKI repositories are distributed with rsync-based synchronization with RRDP (RPKI Repository Delta Protocol) protocol. Also, this chapter introduces the part of update discovery, session_id and serial handling, snapshot and delta synchronization, and hash and schema validation. In contrast, this chapter illustrates an overview of the RPKI validator architectures and their interaction with routing systems.

3.2 Inter-Domain Routing System

The Inter-domain routing system is more about how the networks on the Internet communicate with each other. Those networks are called an Autonomous Systems (ASes) [21]. There are two main parts in any network: end devices, where computers and servers send and receive IP packets and are identified by IP addresses, and routers that direct data packets along the correct path to reach their destination.

As described earlier, BGP lacks a mechanism to verify the origin of the Autonomous System (AS) that is actually authorized to announce a given prefix [22]. To validate the prefixes announced by the ASes, the Resource Public Key Infrastructure (RPKI) was introduced. It provides cryptographically signed Route Origin Authorisations (ROAs) that specify which ASes are allowed to announce specific IP prefixes.

3.3 Resource Public Key Infrastructure (RPKI) Architecture

3.3.1 What is RPKI and how it works in the Internet

The Resource Public Key Infrastructure has become a keystone for security, especially for providing secure inter-domain routing, and a mechanism to validate whether an Autonomous System (AS) is authorized to announce a specific IP prefix.

In today's architecture, the RPKI combines Internet number resources (INRs), such as IP prefixes and AS numbers, with cryptographic keys. It follows the same hierarchy structure as Public Key Infrastructure (PKI). RPKI creates a cryptographically signed message and says which Autonomous System (AS) is allowed to announce IP prefixes [23]. Also, allows the router to check whether the announcement should be trusted or not by checking those messages.

As it follows the Public Key Infrastructures (PKI) hierarchical architecture, the chain starts with the Internet Assigned Numbers Authority (IANA), which allocates IP addresses and AS numbers to the Regional Internet Registries (RIRs). Which are ARIN, RIPE NCC, APNIC, LACNIC, and AFRINIC. Afterwards, the network operators receive them from these regional registries. A digital certificate is issued whenever resources are handed over to verify the ownership and associate them with a specific public key.

These RIRs act as certificate authorities (CAs) and issue certificates that enable the feature to create signed RPKI objects, mainly ROAs, that say which AS is allowed to announce which IP prefix [23]. Then these signed objects are stored in the RPKI repository, from where RPKI validators or relying parties (RP) can fetch them. The other objects are the manifest, the certificate, and the certificate revocation lists (CRLs), where the manifest lists elements that should be present in the repository, whereas the CRLs mark certificates that are not valid.

Route Origin Validation (ROV) is the main way RPKI is used. In ROV, a router compares validated data (VRPs) with a received BGP announcement, and the announcement is categorised as valid, invalid, or not found based on whether a matching authorization exists. To reduce the possibility of a malicious prefix, the network provider can apply routing policies such as rejecting invalid announcements or deprioritizing them.

The hierarchical trust chain in RPKI is like a parent-child relation. In the trust chain, it starts with the Internet registries, and it passes down the permission to larger and smaller networks in the tree. The RIRs are the ones who publish the information that

needs to be verified and signed during the validation process. When the repository data are downloaded by the validators, they check the certificate chain, the cryptographic signatures, and the manifest to see if any files are missing, and they use the CRL to find untrustworthy certificates. Once the validation is done, ROAs are transformed into output in the form of Validated ROA Payloads (VRPs), which describe the trusted link between a prefix, a maximum prefix length, and an origin AS. Those VRP lists are sent to the router. If any problem is found in the repositories or in the validators, it can affect the router decisions.

The RPKI has two main components in it. They are:

- **certificates and signature:** The resource holder required certificates and tools to create an object with prefixes and ASNs.
- **Origin Validation:** To validate those prefixes and the ASNs and to compare them with the BGP table, some tools are required.

3.3.2 Certificates and Signature:

The certificates are used for the cryptography validation when fetching RPKI data. The repositories are accessed by using Https connection. Mostly, the preinstalled root certificates are used, and they come with the operating system or with the browser. They are not specified for the RPKI.

Trust Anchors (TA)

There are some global organisation that manages the Internet number resources, such as IP addresses [24], Autonomous system (AS), and they also provide specific blocks for the resources. There are five resources (ARIN, RIPE NCC, APNIC, LACNIC, AFRINIC) called RIRs, Regional Internet Registries [24].

They are responsible for providing IP addresses and AS numbers. As they act as the top authority, they are referred to as the Trust Anchor (TA) in RPKI. This lets the owner know that the IP address prefixes and the As number belong to the user/owner by signing their private key. Publish that information so that the validator can check and verify that.

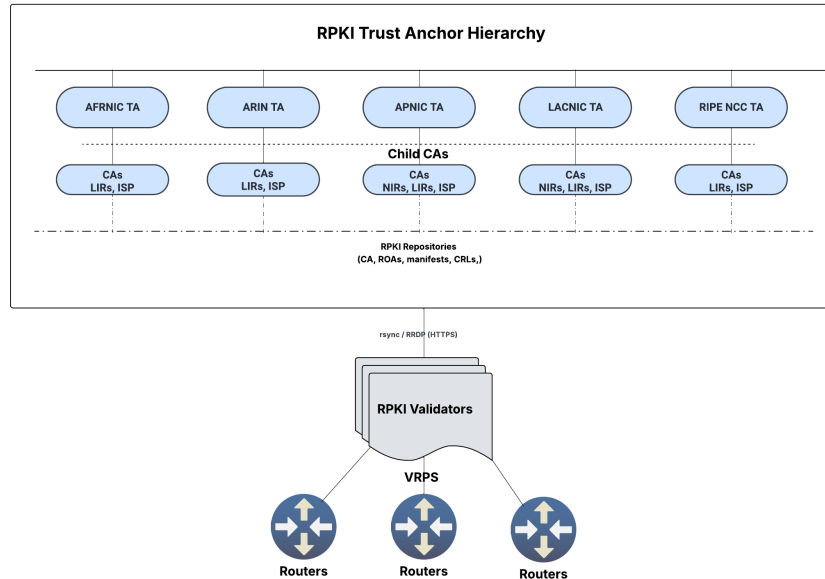


Figure 3.1: RPKI trust anchor hierarchy showing the five Regional Internet Registries (RIRs) and their certificate-based relationship to the IANA root trust anchor.

All the RIRs' resources use their own RPKI repository that publishes the digital certificate, the prefixes, and the ASNs. These objects can be fetched over the internet. To authorize an AS to route prefixes, the network needs to get the signed certificate from its RIR.

Route Origin Authorization (ROA)

There are some data tables that the resource holder uses to log in. This data table is offered by the RIRs, and it also helps to find the ASN and prefixes that belong to them. They create a signed object using a public key. Those objects are authorized ASN, prefix, and the maximum length of the prefix. These signed objects are called ROA. It tells which AS can announce the prefix. If we control the ASN and the prefix at RIR, it is possible to revoke the ROA by creating, deleting and or changing the ROA objects.

3.3.3 Route Origin Validation (ROV)

The second part of the component in RPKi is to validate the ROAs. It checks the ROA objects. According to its hierarchy, the root certificates point to its subordinate certificates so that all RPKI data can be accessed. All the RIR Trust Anchor (TA) files are stored in the Trust Anchor Locator (TAL). This tells the location of the root certificates to the validator during the validation process. This TAL file consist the public key and the URIs of TAs certificate so that they can be fetch.

When it starts fetching data objects from the repository, it marks routes as valid, invalid, or not found.

3.3.4 RPKI Validators

Now, to verify the ROA, certificates, public key, and other objects, we use RPKI validators. Instead of checking them on the router's control plane, RPKI validators are built. These validators are software packages that have all TALs and all RIRs.

Basically, when the validation starts, the validator initially looks for all TAs and downloads all the certificates and ROAs. The RPKI validators act as a local cache and process them all. Whenever it validates the ROA, it creates an object with an IP prefix, maximum length, and origin AS number. In recent days, the RPKI validators use the RPKI Repository Delta Protocol (RRDP) instead of rysnc. Those validated ROA objects are called the validated ROA payload (VRP).

Whenever the validators process ROA, it checks the integrity, authenticity, and digital certificates. Once the ROA is validated, it extracts the data into VRPs list. This VRP has many text formats, such as JSON, csv, OpenBGPD, and BIRD.

3.3.5 Routers

After finishing the validation, those vrps lists are downloaded by the router. The router uses the RPKI router protocol (RPKI-RTR) to download from the validator cache [25] and [26]. Afterwards, the router compares those lists against the BGP table. As mentioned earlier, it has three states of declaring the validation (valid, invalid, and not found).

- **Valid:** When the router finds an announced prefix that matches the VRP's AS, it updates it as valid. It means the original announced prefix is equal to or more specific than the VRP's prefix. Importantly, the length of the announced prefix is not longer than the VRP's maxLength.
- **Invalid:** It is possible in two cases. One is that the announced prefix is different from the VRP's list. The other one is that the announced prefix is more precise than what is allowed by the VRP list.
- **not found:** The prefix is missing in the VRPs list.

3.4 Understanding of RPKI Repository Delta Protocol (RRDP) working flow

This section discusses the significance of the RRDP protocol and how the validators use it to fetch the RPKI repository. As the router cannot directly fetch the RPKI objects, it uses validators to fetch data.

3.4.1 What is RRDP?

Before starting with RRDP, the RRDP stands for RPKI Repository Delta Protocol. This protocol was built by the Internet Engineering Task Force (IETF) to deliver an update on the state of the RPKI repository. The RRDP protocol established the connection between the validator and the repository server using Https. The protocol is mainly designed for scaling and incremental updates of the RPKI repository states.

Initially, the RPKI repositories were just directories that were accessible via an rsync server. During the validation, the validators used to sync the whole folder in one go and download all the CA's repository with one rsync. In RRDP, it does not fetch the entire repository directly. Instead, the CA repository is hidden now behind the RPKI Notify URI. Follows this notify URI path to find the data. Also, this CA Repository section inside the certificate's SIA extension provides two different meanings. One of them could be a real rsync URL path, or the other one could be just a hint for a relative location inside the global RPKI directory structure. These things can create complications or make the implementation complex.

3.4.2 How a validator discovers RRDP

The resource certificates have a unique field called Subject Information Access (SIA). It provides access to the RRDP repository and notifies the validators about the RPKI data location, such as CRLs and ROAs.

Below is a clear explanation of how validators find RRDP:

1. The SIA Extension: Every resource certificate issued by a Certificate Authority (CA) contains Subject Information Access (SIA) extensions. These extensions act as a "signpost." It means that if a CA is using an RRDP, then it should add the link in the SIA extension to find the repository location. This SIA directly points to an Update Notification File instead of the whole directory, which is the starting point of the RRDP protocol.

2. Object Identifier OID: The protocol uses a unique Identifier to differentiate the RRDP pointer and the rsync methods. Object identifier has some structure that can be defined as, id-pkix is as 1.3.6.1.5.5.7, id-ad (Access Description) as id-pkix 50 , and the id-ad-rpkiNotify as id-ad 15 . Lastly, the full OID looks like 1.3.6.1.5.5.7.48.13

3. Https URL: After the OID identification, the validators immediately look for the accessLocation. This location is an HTTPS URI that directly points to the Update Notification File and establishes the TLS Transport Layer Security (TLS) connection to secure the repository. This repository server holds objects like certificates, CRLs, etc., for that specific CA.

The workflow: When the validator starts processing a certificate, it starts with parsing the SIA extensions of that certificate. After that, it looks for the **id-ad-rpkiNotify OID**. Once the validators find it, it extracts all the Https URI. Then, from that URI, the validators download the "Update Notification File." This file has a unique identifier number called "Session_id." Also, this file includes the link for the "Snapshot and Delta" files that need to synchronize the local cache and the session_id.

3.4.3 Definition of RRDP files

The RRDP protocol has three XML files. These are the Notification file, Snapshot file, and Delta files. During the validation process, the validator starts by fetching the notification file. The notification file contains the metadata of Snapshots and Delta files

RRDP XML Files Structure

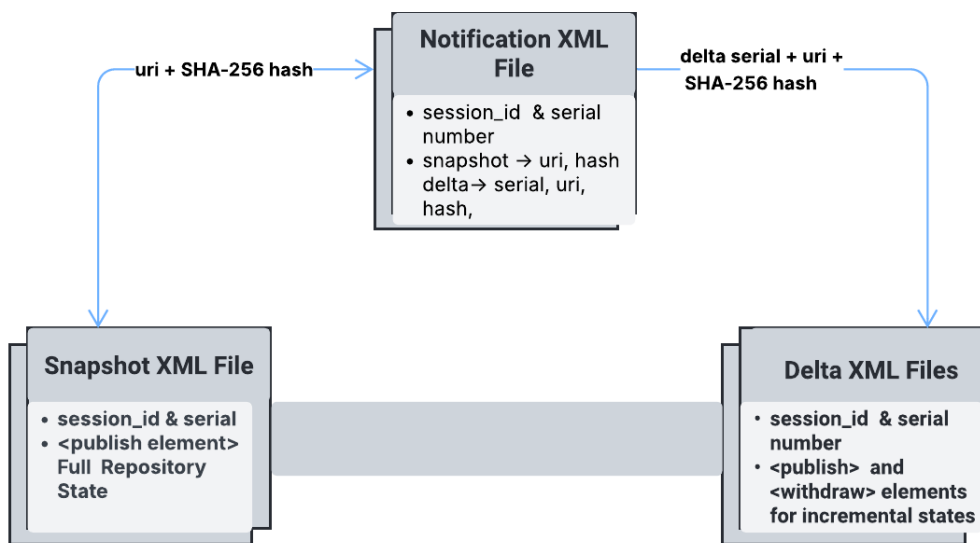


Figure 3.2: Structure of RRDP XML files and their relationships. The Notification.xml file references both the Snapshot.xml file and the Delta.xml files via URIs and a SHA-256 hash value.

Notification File: In the RPKI Delta protocol, the notification file acts as the entry point. This file is the one that tells the validator about the latest version of the repository, including the location to fetch it. The notification file is a file that changes over time. It provides the current state of the repository, such as `session_id` (current session), `serial-no` (current version of the repository), hash and the links to the latest file location. As this file is dynamic, it can not be cached for more than one minute.

Snapshot File: The snapshot file, it is the complete state of the repository at a specific point in time. This file is for initialization, which is the first synchronization or fallback to the snapshot file if the delta fails. The snapshot files are immutable once it published with a specific session or serial id and URL. Because of its static behavior, it can be cached indefinitely.

Delta files: This file contains only the changes or the incremental updates between two serial numbers. It publishes actions like adding or updating an element, whereas the withdrawal actions are deleting objects from an element. The delta files are like a snapshot file, which is immutable and can be cached indefinitely.

3.5 Working Model of RRDP protocol

This block explains the mechanism of the RPKI Repository Delta Protocol (RRDP). The RRDP protocol mechanism operates within a structured synchronous loop. This synchronization loop process keeps all of the validators or the Relying Parties (RP) repository cache consistent with the remote server. The loop starts with fetching an Update Notification File by RP. This Notification File contains the current version of the repository, such as the session ID and serial number.

After fetching, RP checks the session id if the session has not changed and all the delta files are available, RP performs an incremental update to refresh its cache. If the session id does not match the local state or some of the delta files are missing, the RP will update the local state completely by downloading the full Snapshot File.

Basically, this logic is to keep the cryptography consistent while minimizing the data transfer. The RRDP protocol relies on three core mechanisms, as shown in the figure below:

RRDP Working Model

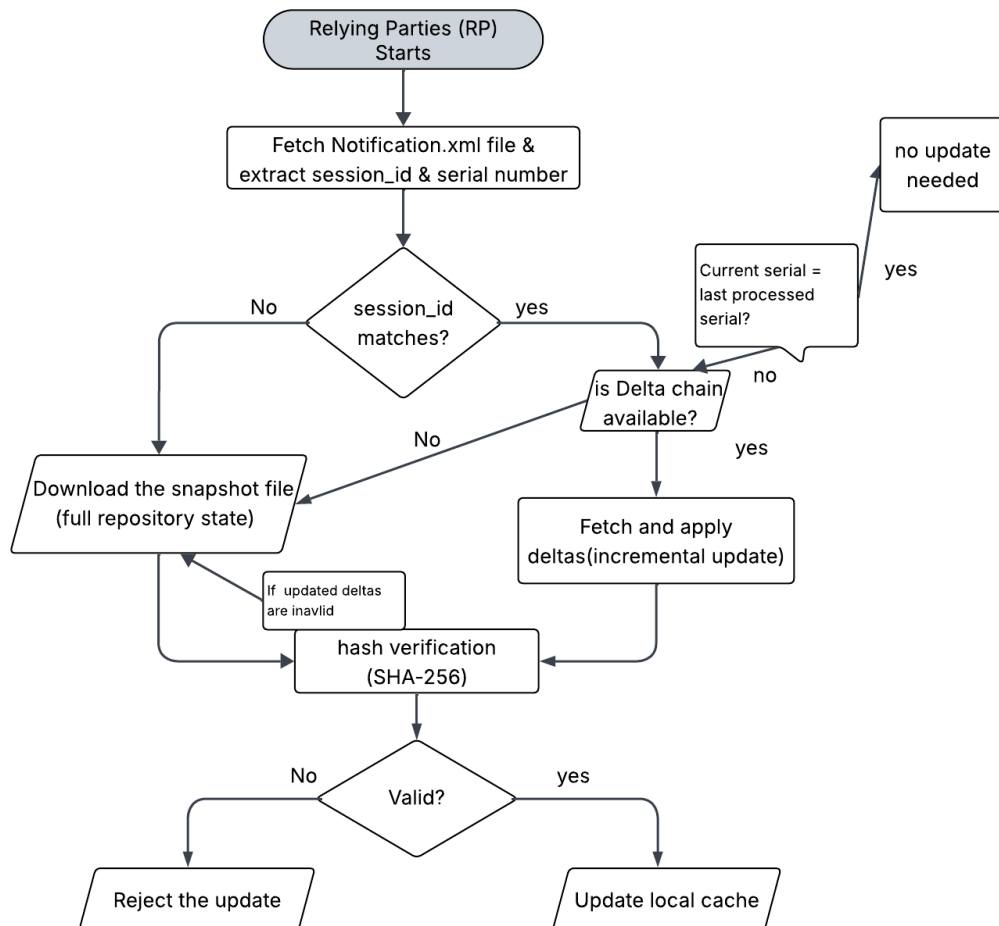


Figure 3.3: RRDP XML files synchronization workflow from the point of view of a Relying Party. It describes how to handle sessions, choose between delta and snapshot, and check hashes.

3.5.1 Polling and Tracking

This polling and Tracking is the first stage of the RRDP protocol synchronized loop. The validators start with fetching the Update Notification File from the repository. This XML file refers to two essential values:

- **Session_id:** This contains a unique ID, which is a UUID. It identifies the current elements of the repository.
- **Serial Number:** A Serial Number steadily increasing integer. It identifies or marks the new version of the repository state of the current session.

The validator checks whether an update is required by comparing the session_id and the serial number.

3.5.2 Synchronization Path Decision

The second stage is path selection after evaluating the Update Notification File. There are two paths to synchronize its local cache, but the validators choose one of the following based on the current state:

- **Incremental Path (based on Delta):** Whenever the session_id matches and the continuous session of Delta Files is available, the validator uses incremental updates. This is more efficient as it minimizes the process time.
- **Cache Refreshing (Snapshot):** This path is chosen when the session_id has changed, or some deltas are missing. In those cases, the validators perform a full cache resynchronization, where it downloads the complete Snapshot file that contains the current state of the repository and update the cache entirely.

3.5.3 Integrity and Consistency Validation

Regardless of which path was used, RRDP requires strict cryptographic security checks. The validators must verify the following things before applying any data to its local cache.

- **Hash Check:** The hash SHA-256 value listed in the Update Notification File must match exactly with the downloaded delta or snapshot file.

-
- **Delta Match:** In the delta match, the `session_id` and serial number inside the downloaded file should match what the Update Notification File states.

The deltas-first-when-available it is the main approach. This rule ensures that the validators keep an exact, cryptographically secure copy of the repository.

3.6 Challenges in RRDP implementation

There are many complexities in implementing RRDP. This section describes the challenges in developing the RPKI delta repository.

Inconsistencies may arise from a misconfigured server or cache as the RRDP relies on the Https request. It updates the Notification, snapshot, and the delta files. The validator may retrieve an updated notification file from one backend that refers to the delta files, which do not exist on the other backend. This happens as the RPKI operators use https load balancers in front of the RRDP backend server, which is not properly synced.

RRDP uses HTTPS to transport unsigned XML. Even though RPKI objects are cryptographically secured, the files still can be modified during delivery [27]. Furthermore, the RRDP should treat the RPKI data as malicious until it validates objects. It should not apply any actions immediately, such as withdrawals, without additional checks.

To avoid conflicts between the RRDP sessions or repositories, validators should create a separate file name for the valid and invalid data. RRDP has a file name system issue in implementation [28].

4 Methodology & Approach - Implementation of RRDP Differential Fuzzer Framework

This part describes the methodology and implementation of RRDP-Diffuzz, which is a differential fuzzing framework designed for RRDP-based RPKI validators.

The goal of this framework is to discover the behavioral differences between two or more independently built validators when they process the same RRDP repositories. This framework examines multiple RPKI validators by feeding both a valid baseline and a mutated input to observe the resulting outcome, which includes success or failure states, VRPs, and the pattern of error.

The divergences in validators' behavior upon identical inputs are treated as potential analysis for the future.

This block represents RRDP-Diffuzz, a Differential Fuzzing tool designed for exposing inconsistencies in RRDP-based RPKI validators. The system automatically generates valid RRDP XML files. It then copies the generated data into a target directory, serves them via a local Nginx RRDP, and executes multiple validators against the input data.

In this implementation, the core concept is XML mutations; it applies mainly structure-aware mutations to RRDP XML files and generates both valid and invalid RRDP files.

For each fuzzing loop, those mutated repositories are then fed to multiple validators as black boxes to observe and compare their behavior or any bugs.

This design basically follows the CURE fuzzing approach for RPKI but focuses on RRDP processing as specified in RFC 8182.

4.1 The Fuzzing Workflow

The Fuzzing framework executes in a repeated loop to find the behavioral differences between multiple RPKI validators. A fixed and deterministic pipeline is essential to guarantee that any observed behavioral differences come from validator implementations rather than changes in execution or test cases [7]. Each loop follows a fixed pipeline so that results are comparable and reproducible. This workflow has the following stages:

Baseline Validation: The system runs all the validators against the original, unmodified repository before it starts fuzzing. The main idea is that all the validators should produce the same result when reading a clean, valid repository [1]. The output is saved immediately as a reference for all comparisons during the fuzzing run. If the original files are accidentally modified during fuzzing, the recorded SHA-256 hash of the original file will detect it.

Preparation of Test Cases: For every iteration, the fuzzer runs a new test case and makes a fresh copy of the original file. This ensures that one test does not affect the other. All the test gets a random unique number based on the iteration number and the present time. This unique number decides what changes were made to the repository and saves them to the disk so that the test case can be repeated later to reproduce the same result.

Mutation Phase: There are two phases in mutation. In the first phase, the fuzzer applies a stage 5 modification on the snapshot XML file. The notification file updates its hash, serial number, and session_id after the mutation is applied to the snapshot file. To keep the repository consistent, all the attributes are kept in sync.

In the second phase, right now it runs 30 % of iterations. In this stage, the fuzzer makes sure that the snapshot reference is still valid when applying targeted attacks to the notification file. It lets the fuzzer know how the validators behave when they handle the abnormal notification level behavior without breaking the repository structure.

Serving: At this stage, the mutated repositories are served through a local nginx HTTPS server, which operates under an authentic network that exactly matches the actual RRDp protocol system. At the start of each iteration, the symlink is updated and points to the current mutated repo so that nginx can always serve the test case without a restart.

Execution: In this part, for each iteration, all the validators are run against the mutated data independently. To ensure that there is no old data left from the previous runs [11]. The validator runs with disabled rsync, a fresh cache, and temporary TAL files. It avoids interference in current results. The validators only care about the output each one produces, not how they work internally [7]. The output from each validators converted to a standard format before doing the comparisons. A hash value is computed from that output, and then compared across validators. If it does not match it record the case as a finding.

Oracle Dashboard: After running all the validators, Oracle compares the output results that are saved as test cases. Basically, it provides a comparison table of all the validators that accepted or rejected the same routes. It also shows that accepting the same input but producing different routes is Type A. Type B is one validator accepted, and the other one rejected. After the run, the system also verifies whether the XML was well-formed. If any valuable divergences are found, it will save the mutated input and the generated output to the disk so that it can run repeatedly.

4.2 Approach 1 - Structure-Aware XML Mutation

4.2.1 Overview

Furthermore, this section describes a standalone XML_mutator crate to mutate RRDp XML files in a controlled environment for differential testing across multiple RPKI validators [7]. This contribution also examines whether the built fuzzer is stable in detecting inconsistencies and how it applies mutations to XML files while keeping the XML structure well-formed.

Every XML file is handled by the XML_Mutation crate as an AST tree structure, similar to a family tree [29]. In this tree, each element has a node connected to its parent and its child nodes. This structure shows how the XML is stored in memory during execution.

These mutations are designed to systematically test the validator’s assumptions on the structure, reordering, and attribute handling by applying controlled modifications to the XML tree.

A mutation is defined as a transformation function, $\mu : T \rightarrow T'$, which maps an input XML tree T to modify it as a tree T' . Every mutation slightly modifies the XML structure. The mutator will show the sibling elements in each loop, add or remove nodes, duplicate the current element, or alter tag values or attributes. This execution will occur in an XML tree.

The engine can apply multiple mutations one after the other in a single fuzzing loop. The resulting document is produced by combining all transformations in sequence -

$$T' = \mu_k \circ \mu_{k-1} \circ \dots \circ \mu_1(T)$$

The use of a fixed random seed guarantees deterministic behaviour, ensuring reproducible mutation results, even though each mutation may choose at random which nodes to modify.

4.2.2 Mutation Pipeline

The `xml_mutator` crate applies mutation in five sequential stages. Each stage perform mutation on the target node or element of the XML files. The mutation starts with applying a permutation to the structure-Aware Mutation to XML attacks.

These stages were chosen based on the phase-by-phase design of the mutation process. The main purpose was to ensure that most iterations were processable by the validators, while introducing URI, hash, and XML-level perturbations to explore inconsistencies.

Permutation or Child Reordering: The mutation starts with shuffling the order of the elements within the XML tree. Even though [3] allows XML elements to be in any order, this test is to check whether the validators depend on element order. The system shuffles targeted tags like **Publish** and **Withdraw** elements based on the XML file type, whether the file is a snapshot, delta, or notification. Per iteration, there are multiple rounds of permutation to produce many test cases to find if validators fail when sees unexpected order of elements.

Example of Reordering:

Before applying shuffling in (delta):

```
<delta serial="15">
  <publish uri="../../../a.mft">...</publish>
  <withdraw uri="../../../b.roa" hash="..." />
</delta>
```

After suffering siblings:

```
<delta serial="15">
  <withdraw uri="../../../b.roa" hash="..." />
  <publish uri="../../../a.mft">...</publish>
</delta>
```

Structural Mutations:

- **Tag Renaming:** This mutation checks if a validator depends on specific RRDP element names instead of relying on strong schema-based validation. A publish element of the delta file is renamed to withdraw if it exists. The renaming happens without affecting its attribute, child element, or its location in the XML hierarchy. As a result, the validators are forced to process unexpected data while keeping the overall XML structure intact. Validators that assume strict tag naming may show inconsistent behavior with this mutation.
- **Attribute Modification:** Attribute Modification targets how the validators interpret metadata in RRDP XML files. The mutation modifies the value of an existing attribute on the selected XML element of the notification file when keeping the hierarchical structure and element text unchanged. This operation is effective to check whether the validators check the attribute status correctly.
- **Inserting Child Element :** The child element insertion tests how well validators handle extra child elements into the existing XML tree without breaking the structure. The function inserts a new fake publish URI element into the existing snapshot XML file. If there are any delta files, it inserts a new publish or withdraw element with random attributes. which ensures syntactic validity. Even though the document remains well-formed, it may still break the RRDP rules and show the inconsistency in validator behavior.

-
- **Child Element Deletion:** The child element deletion function deletes the chosen element from the XML file without breaking the valid XML tree structure. By removing an essential element from the tree, it tests how the validators behave when an expected element is missing from the tree.
 - **Child Element Duplication:** The duplication mutation copies unnecessary structural elements and adds them within existing XML elements. This mutation applies in the notification file to see the validator's performance, especially when it duplicates the existing delta reference, if available, or inserts a new delta element with a random serial number, URI, and a hash value.

URI Modification: This is stage three, notification URI testing. This stage applies modification into snapshot and delta URIs with URLs that cannot be reached to test how validators handle broken links. The system uses the `is_manifest_bound()` function to avoid changes in the Publish and Withdraw elements to ensure modification only on the notification level rather than crypto edits. It maintains the XML structure's integrity. This targeted mutation can be applied with an adjustable probability. Currently, the system is running with a probability of 20 % per iteration.

Hash Edits: In this stage, this mutation applies a hash manipulation by changing a random hexadecimal character in the notification file. This test specifically targets the snapshot and available delta hashes to see if the validators correctly validate the downloaded file against the hash that was provided in metadata. Unlike URI edits, this mutation also avoids modifying the publish or withdraw element to maintain the RPKI objects' validity. The changes occur with a probability of 5 % of all fuzzing cycles.

Attacks on XML: Lastly, to identify a divergence between validators, this stage applies eight different types of XML mutation (CDATA, comment injection, unknown attributes, namespace, inserting whitespace in metadata, and deep element wrapper). This mutation preserves XML files as logically and structurally valid when introducing semantics variations. Basically, they are implemented to bypass simple checks and reach deeper application logic, which can expose the weakness of the XML processing across different validators [8]. In the default configuration, these attacks are applied with the probability of 45 % per iteration.

4.2.3 Recursive Tree Traversal

In this section, it recursively applies a mutation using a traversal function. This function manages how the mutations are distributed across the XML tree. The `for_each_elem_mut` function ensures that the stages three, four, and five are not limited to any level; it can be applied across every element within the XML tree. To increase the chance of finding a parsing inconsistency, the fuzzer applies a URI edit, hash edits, and inserts attacks on XML by targeting the entire structure.

4.2.4 Protocol-Guided Mutation Decisions

To guide the mutation behavior, first, the mutation engine checks the document type, whether the input is a notification, a snapshot, or a delta file based on root elements, and applies mutations accordingly. There are published elements in the snapshot file. Similarly, there are publish and withdraw elements in the delta files, and at least one reference in the notification file snapshot or delta. They are all reordered in stage 1.

In stage 2, there are structural edits per document type. Whereas stages 3 and 4 applies notification level URI and hash attributes edits without touching publish or withdraw elements. As a result, the mutation focuses on how the protocol works and the structural rather than semantic mutations. Therefore, the following algorithm 1 focuses on the RRDP XML files' structural mutations rather than semantic content changes.

Algorithm 1 Protocol-guided RRDP XML mutation

Require: X : RRDP XML input, C : mutation configuration, s : seed

Ensure: X' : mutated XML output

```
1:  $T \leftarrow \text{ParseXML}(X)$ 
2: if  $T = \perp$  then
3:   return  $\text{FallbackStringMutation}(X, s)$   $\triangleright$  Handle unparsable input
4: end if
5:  $k \leftarrow \text{RootName}(T)$   $\triangleright$  Detect document type
6:  $h \leftarrow \text{FNV1a64}(X)$ 
7:  $\text{rng} \leftarrow \text{InitRng}(s \oplus h)$   $\triangleright$  Deterministic randomness
8: for  $i \leftarrow 1$  to  $C.\text{stage1\_permutation\_rounds}$  do
9:    $\text{PermuteChildren}(T, k, \text{rng})$   $\triangleright$  Document-specific reordering
10: end for
11: if  $C.\text{stage2\_structural\_edits}$  and  $\text{Bernoulli}(\text{rng}, C.\text{stage2\_probability})$  then
12:    $\text{StructuralEdits}(T, k, \text{rng})$   $\triangleright$  Document-specific structural edits
13: end if
14: if  $C.\text{stage3\_uri\_edits}$  and  $\text{Bernoulli}(\text{rng}, C.\text{stage3\_probability})$  then
15:    $\text{MutateUriAttrs}(T, \text{rng})$   $\triangleright$  Skip manifest-bound elements
16: end if
17: if  $C.\text{stage4\_hash\_edits}$  and  $\text{Bernoulli}(\text{rng}, C.\text{stage4\_probability})$  then
18:    $\text{MutateHashAttrs}(T, \text{rng})$   $\triangleright$  Skip manifest-bound elements
19: end if
20: if  $C.\text{stage5\_xml\_surface}$  and  $\text{Bernoulli}(\text{rng}, C.\text{stage5\_probability})$  then
21:    $\text{ApplyXmlSurfaceAttack}(T, \text{rng})$   $\triangleright$  Apply one XML surface mutation
22: end if
23:  $X' \leftarrow \text{SerializeXML}(T, C.\text{output\_style})$ 
24:  $X' \leftarrow \text{PostProcessSentinels}(X')$   $\triangleright$  Materialize comment, CDATA, and PI injections
25: return  $X'$ 
```

This Algorithm 1 presents how `xml_mutator` applies mutations to the XML structure in several stages. It takes time to apply mutation in phases, but it also depends on the iteration runtime, which is influenced by the validator's cache preparation, execution, and repository serving.

4.3 Approach 2 - Overview of Orchestrator Workflow

This block describes how the main system orchestrator was built and how the RRDP validation process tests the validators. The orchestrator controls the whole experiment from the start to the end of the validation process.

Initially, the system runs all the selected validators against a valid RRDP repository. This baseline validation run creates a reference result that is used later for comparison with mutated data. If any of the validators fail at the baseline verification stage, the fuzzer stops the validation process immediately.

Afterwards, if the baseline test passes, the orchestrator runs in a loop where the validation continues until the system reaches its targeted time limit. The system records all abnormal findings during each iteration. It creates a fresh copy of the repository in each iteration and applies mutations to the snapshot file and, in some cases, to the notification file. Based on the mutation applied to the snapshot file, the notification file updates its hash, session_id, and serial number so that the validators do not fail at the early stage of the validation process.

Then the mutated repository is served to all validators. The validators parse these crafted repositories and generate outputs. The results are normalized and sorted to detect differences between the validators.

The framework checks whether the validators agree on the generated output at the end of each round. There are two types of disagreement the orchestrator looks for. The first is an accept/reject split, where some validators accept the crafted input while others reject it. The second is a VRP set divergence, where all validators accept the mutated repository but generate different VRP lists. Not all findings are counted as genuine differentials; the system first filters out confounded results. For example, if the XML files are already broken or rpki-client falls back to rsync or cache instead of using RRDP, those findings are excluded from the clean differential count. Relevant differential cases are saved to disk and can be replayed or reproduced later.

The next page illustrates the algorithm of the overall differential fuzzer workflow.

Algorithm 2 Differential fuzzing orchestration

Require: R : live RRDP repository, V : validator set, τ : runtime limit

Ensure: Differential findings and saved test cases

```
1: SetServedRepo( $R$ )                                ▷ Expose baseline repository
2:  $B \leftarrow \text{RunValidators}(R, V)$                 ▷ Baseline validation
3: if AnyFailure( $B$ ) then
4:   return                                           ▷ Stop if baseline is invalid
5: end if
6: StoreBaseline( $B$ )
7:  $f_b \leftarrow \text{Fingerprint}(R)$ 
8:  $i \leftarrow 0$ 
9: while ELAPSED TIME() <  $\tau$  do
10:   $R_i \leftarrow \text{CopyRepo}(R)$ 
11:   $s_i \leftarrow \text{MakeSeed}(i)$ 
12:  MutateSnapshotAndPatchNotification( $R_i, i, s_i$ )
13:  if NotificationMutationSelected( $i, s_i$ ) then
14:    ApplyNotificationMutation( $R_i, s_i$ )
15:  end if
16:  SetServedRepo( $R_i$ )                                ▷ Serve mutated repository
17:   $O_i \leftarrow \text{RunValidators}(R_i, V)$ 
18:  if FINGERPRINT( $R$ )  $\neq f_b$  then
19:    return                                           ▷ Abort on baseline corruption
20:  end if
21:   $D_i \leftarrow \text{DetectDifferential}(O_i)$ 
22:   $T_i \leftarrow \text{Triage}(R_i, O_i)$                 ▷ XML and transport checks
23:  if  $D_i$  then
24:    RecordFinding( $i, s_i, O_i, T_i$ )
25:    SaveCase( $i, R_i, O_i$ )
26:  end if
27:   $i \leftarrow i + 1$ 
28: end while
29: SetServedRepo( $R$ )                                ▷ Restore baseline repository
30: PrintSummary
```

Algorithm 2 shows the end-to-end fuzzing loop, from copying the repository and applying mutations to serving the mutated repository to the validators.

4.4 The Orchestrator Architecture

The previous section describes the overall workflow of the orchestrator during the RRDP validation process. This part will explain the pipeline of how validators start validating the valid baseline, followed by applying `xml_mutation` crate, and execution of multiple validators together, including analyzing their inconsistent behaviour. Since the XML mutation part has already been explained in detail earlier, this part only summarizes the steps in the fuzzer workflow.

4.4.1 Baseline Validation

Following the Figure 4.1, initially, the orchestrator starts the validation process by validating the valid unmodified RRDP repository. The idea of this test is to run the validators against the valid repository before the modification and save the results for reference. If any differences are observed after applying the mutation, it will only be valid when the validators behave consistently upon feeding a valid original baseline.

To test this validation, the system first points the served directory `rrdp_served` to the live baseline repository named `data/rrdp`. It creates a temporary TAL directory and initializes the Oracle to store data. After that, the fuzzer runs all the selected validators against the valid repository and collects their results in the chosen format. These baseline results are used to determine whether the fuzzer can continue the validation process.

Additionally, whenever the validator produces a set of non-zero VRP lists from a valid input repository, it is considered successful validation. If any of the validator fails the baseline validation check, the fuzzer stops immediately, even without starting the mutation iteration.

Once the validators pass the baseline validation, the system starts storing the output results in a normalized format with a metadata file that contains its log history on exit status, VRP's count, acceptance status, and hash.

Differential Fuzzer Workflow

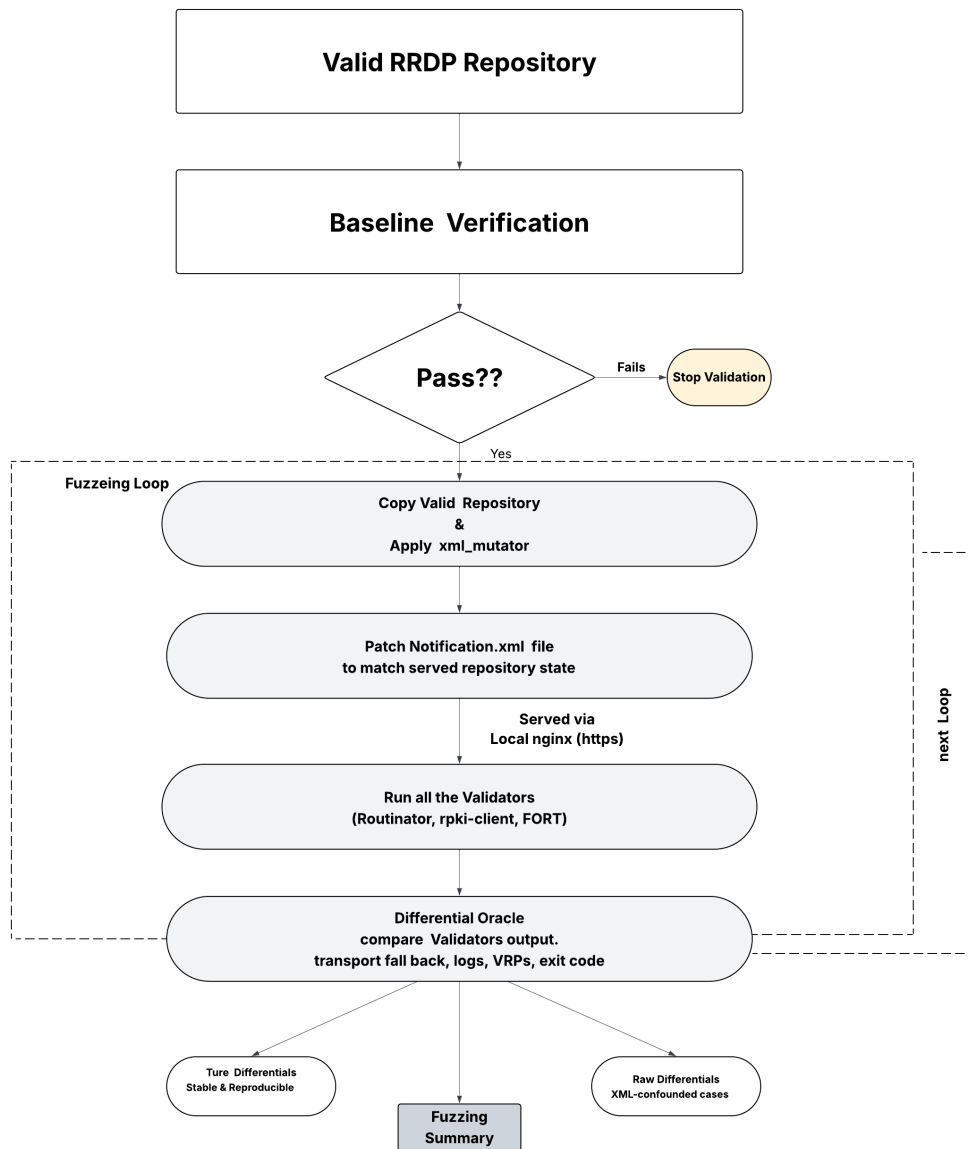


Figure 4.1: Overview of a Differential Fuzzer Work model, showing baseline verification, mutation loop, multi-validator execution, and differential oracle filtration.

4.4.2 Mutation Strategy

The orchestrator applies mutation on RRDP files in a structured way based on `xml_mutator` crate during each iteration. Before applying any mutation, the system first prepares a fresh copy of a repository and generates a seed for reproducibility. It then applies a mutation on the snapshot file and resynchronizes the notification file so that validators can fetch the mutated snapshot file. Afterwards, the system may apply some semantic attacks to the notification file. The following paragraphs explain the stages of how the orchestrator executes them in order.

Preparation of Each Iteration: As mentioned in the earlier section, the fuzzing loop starts by copying a valid repository into the `data/mutated_rrdp` directory. The seed is generated by mixing the nanosecond timestamp with the iteration number to ensure mutation diversity across iterations. To reproduce any findings from an iteration, the seed is saved as `seed_u64` alongside the mutated files.

Snapshot and Notification Mutation Strategy: After entering a fuzzing loop, the system starts looking for a snapshot file using the URI from the notification XML file. During the first two rounds, only a simple wrap-up mutation is applied. In later rounds, the system applies the full mutation process. After mutating the snapshot file, the orchestrator updates the notification file by copying the `session_id` and serial number from the mutated snapshot. Next, it figures out the correct SHA-256 hash for the changes snapshot file and updates that hash into the notification file. To avoid early rejection at the notification validation level, this step is necessary. Otherwise, the validators never reaches to the snapshot file that the system wants to test.

Why Two Different Steps: The mutation strategies on the `snapshot.xml` and `notification.xml` files were kept separated intentionally. There are a couple of reasons for this approach. Firstly, to reach the mutated snapshot file, the notification file is fixed. Secondly, attacks are applied to the notification file only after the snapshot becomes reachable. Additionally, the separation is also required by RRDP validation semantics, according to RFC 8182 [3], if the hash does not match or if the `session_id` or serial state fails during the validation process, a validator must reject the snapshot or delta file. Before applying these two steps separately, the fuzzer mutated the snapshot and fixed the notification hash, and then mutated the notification file again, but it broke the fixed hash. In this way,

the mutated RRDp input repository gets rejected at the metadata-validation phase before it reaches the core logic of the validators.

Attacks on Notification: After applying the first and second steps, the system applies mutation attacks on the notification file. This uses a random seed and mutation only on notification by changing the session_id, adding invalid links to snapshots, rolling back the serial number, and adding unknown elements in the XML file. After attacking the notification file, it checks if the real reference to the snapshot is still correct, and the hash and URI are still right. If the mutation messed up the link to the snapshot, that notification gets thrown away. This helps to find out how notification validation works under mutation attacks.

4.4.3 Validator Validation Process and Result Unification

After applying the mutation to the RRDp repository, the fuzzer calls all the selected validators to run against the mutated repository. All the validators have their own built-in command line and output format, but the system converts all their results into a data structure called ValidatorVrpRun. This format saves all the output for each of the validators, including their exit status, run time, the list of VRPs, VRPs' hash, and the raw csv output. This makes the validator comparisons easier.

The system started testing with one, then the other validators were implemented in the system. All the validator has their own command line and temporary cache folder.

- In this fuzzing setup, Routinator is executed with the rsync disabled, RRDp custom certificate, and enabled trace logging. The standard csv format output is directly parsed from the resultant output.
- For rpki-client, it uses a temporary cache and output folders and once the validator finishes the process, it reads the generated csv file from the output directory.
- Fort is executed in standalone mode. The validator runs with disabled rsync, and its ROA output goes to standard output so that it can be parsed.

Routinator and FORT are explicitly enforced to use RRDp only by disabling rsync. In the experiment, rpki-client may still fall back when RRDp processing fails. However, for the validator inconsistency comparison, rpki-client's transport fallback is counted as a raw

differential because it may process a different repository state from the current mutated RRDP input.

The orchestrator considers the validator's validation process successful only if it produces output. A validator accepts a repository only if it produces a VRP list with a count greater than zero. This approach is implemented to later check the validators' disagreement on mutated inputs.

4.4.4 Differential Oracle

Subsequently, finishing the whole validation process, the orchestrator analyses the generated results through a differential oracle [10]. The system checks if all the validators agree on the same mutated repository. As explained in the previous section, a validator is counted as accepted only when the generated output is greater than zero $vrp_count > 0$. The system is designed only to look after two types of disagreement. The Differential Oracle operates through some core mechanisms as illustrated in the Figure 4.2

- **Accept and Reject Disagreement:** It is defined when some of the validators accept the mutated repository while others reject it. This classification follows the differential analysis approach used in prior RPKI validation testing [1]
- **Divergence in VRP Set:** When all the validators accept the mutated repository but generate different VRP sets.

If we look at the differential Oracle workflow Figure 4.2 and algorithm 3, whenever the Oracle system finds any disagreement, it saves all the findings in the Oracle directory with the iteration number, baseline, and mutated XML files, type of disagreement, and corresponding hashes.

After checking the XML valid tree structure for both notification.xml and snapshot.xml, it is only counted as real and clean RRDP differentials. Otherwise, it discards the test case as compound differentials.

This differential Oracle Algorithm 3 summarizes how it records validators' inconsistency, cleans them from raw differentials, and stores the final result analysis.

Differential Oracle Flowchart

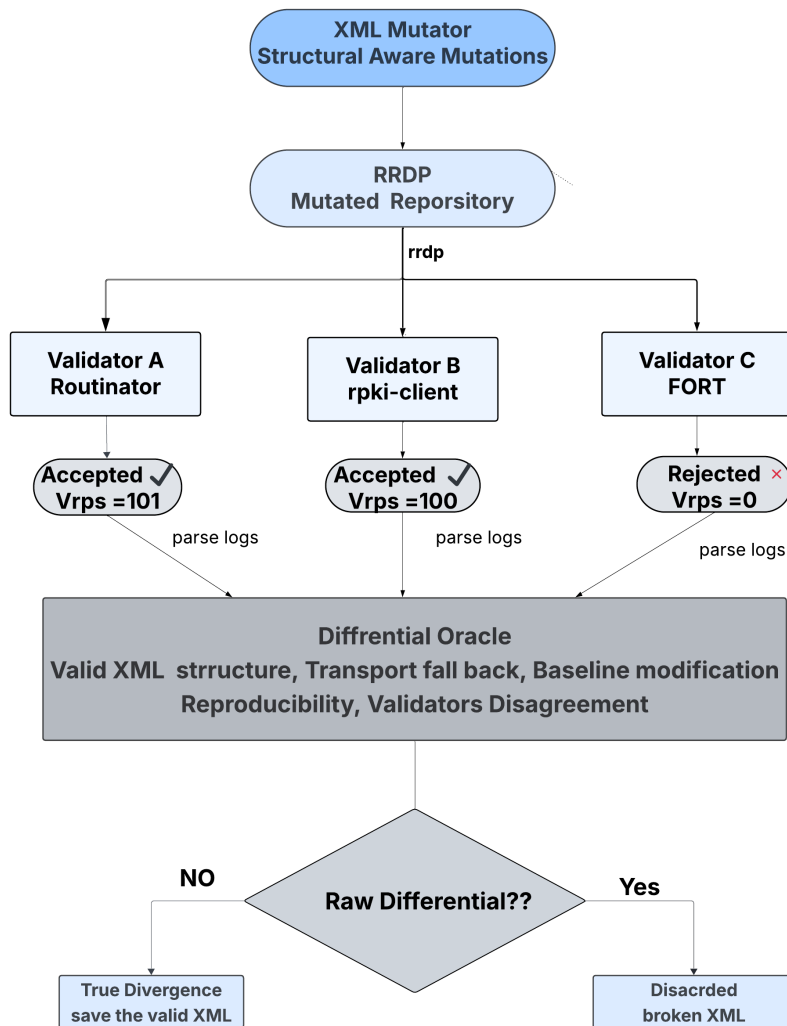


Figure 4.2: Differential oracle workflow for classifying validator disagreements.

Algorithm 3 Differential oracle collection and clean reporting

Require: V : ordered validator names, R : validator runs, F : current findings, m : meta-data ($iter, seed, kind, clean$)

Ensure: stored findings, clean differential table, and raw summary

```
1:  $verdicts \leftarrow []$ 
2: for all  $name \in V$  do
3:   if  $name \in R$  then
4:      $run \leftarrow R[name]$ 
5:      $accepted \leftarrow (run.vrp\_count > 0)$ 
6:     Append  $\langle name, accepted, run.vrp\_count, run.vrp\_hash \rangle$  to  $verdicts$ 
7:   end if
8: end for
9:  $f \leftarrow \langle m.iter, m.seed, m.kind, m.clean, verdicts \rangle$ 
10: Append  $f$  to  $F$   $\triangleright$  Store every finding, clean or confounded
11:  $C \leftarrow \{x \in F \mid x.clean = \text{true}\}$ 
12:  $K \leftarrow |C|$ 
13:  $N \leftarrow |F|$ 
14:  $H \leftarrow N - K$ 
15: if  $K = 0$  then
16:   Print “No clean true differentials found”
17:   Print raw total  $N$  and hidden confounded count  $H$ 
18:   return
19: end if
20: Print clean differential table header
21: for all  $x \in C$  do
22:    $row \leftarrow$  empty string
23:   for all  $v \in x.verdicts$  do
24:     if  $v.accepted$  then
25:       Append “ $v.name : \checkmark (v.vrp\_count)$ ” to  $row$ 
26:     else
27:       Append “ $v.name : \times$ ” to  $row$ 
28:     end if
29:   end for
30:   Print  $\langle x.iter, x.seed, x.kind, \text{yes}, row \rangle$ 
31: end for
32: Print displayed clean finding count  $K$ 
33: Print hidden confounded count  $H$ 
34: Print raw finding total  $N$ 
```

4.4.5 Case Reproducibility

As mentioned in Section 4.4.4, the system saves the identified results in the data/oracle/ directory under a folder named cases/. Basically, a case is stored if any of the validators differ from its baseline reference or if the accepted validators disagree with each other.

Later, the same case can be reproduced by using the same case directory together with the same mutated repository files.

5 Evaluation of Differential Fuzzer

5.1 Introduction

This chapter discusses the metrics that are required to measure the effectiveness of the proposed RRDP differential fuzzing framework. There are three main perspectives on doing these evaluations. First, it measures whether the system can generate a large number of mutated input data and can trigger meaningful behavioral differences across validators. Second, it evaluates the identified inconsistencies across the validators and analyzes what kind of XML mutations lead to divergences in validators. And lastly, it examines the repetition of mutation patterns, including compound differentials, stability and reproducibility of the test cases to avoid false positive results.

This way, this chapter demonstrates the result of the overall quantitative results summary to their detailed explanations.

5.2 Experiment Overview

This section presents the results obtained from the Differential Fuzzing process, which was implemented using the two methodologies that were described in Section 4.2 and Section 4.3. This fuzzing system was built to evaluate the behavior of multiple Relying Parties on mutated RRDP XML files over repeated iterations. In this experiment, the framework was executed several times, with an adjustable runtime. To assess the consistency of the validation process, the system was initially run for one hour, which covered approximately 450 iterations, but the number of iterations is not always fixed for a given runtime; it varies depending on the Relying Party. In the second run, the time was set for twenty minutes, which had around 131 to 151 iterations, while the third one was a shorter run that covered approximately 62 iterations. The main goal of this analysis was to

determine how mutations on the RRDP repository affected the validator behavior during each iteration and to observe whether the validation process showed consistent behavior as the number of iterations increased over time.

To evaluate the behavior of multiple validators, the resulting output was categorized into three parts: the test cases that were accepted by the validators, the test cases rejected by the validators, and the test cases that produced actual differential behavior during testing. After recording all kinds of test cases, the system filters out two types of analysis. One is raw differentials, where every single disagreement is detected between the validators, and the other one is a clean or filtered differential result, where the system re-examines raw differentials and excludes those caused by *rpki-client* falling back to rsync or its local cache, classifying them as non-attributable, and identifies the remaining cases as true differential behavior [1].

This differential fuzzer was tested in an isolated lab, where it first generates the valid RRDP repository, which is later considered as a baseline. During the validation process, the system starts validating the Relying Parties against that baseline. If all the validators pass the baseline check and generate a VRP list, the fuzzer enters a loop and evaluates validator behavior across repeated iterations. For example, the system was executed for 1200 seconds, and it completed 131 iterations in total. If the runtime is set to 1200 seconds and the system starts a new iteration at 1199 seconds, it completes that iteration instead of stopping exactly at 1200 seconds. During these 131 iterations, there were 58 test cases that were accepted by validators, and 17 test cases were rejected. Additionally, there were 45 differentials discovered by the system that were classified as fallback-induced differentials as the *rpki-client* falls back to rsync and then to its cache whenever RRDP fails. At the same time, there were 11 test cases that showed true disagreement between validators.

To evaluate the true disagreement, we have examined the XML structure of the served test cases. Most importantly, during the validation of these 11 test cases, each validator consistently produced the same output across repeated runs, yet the validators disagreed with each other. Basically, Routinator rejected the XML file and produced zero VRPs, whereas the other validators, *rpki-client* and FORT, accepted the same test cases and produced a VRP list. These experiments show that the fuzzer was effectively and repeatedly producing a stable validator inconsistency instead of producing random disagreements.

Similarly, two more analyses on the increased number of iterations are presented in the following sections, where the results are examined across the remaining runs to further confirm this behavior.

5.3 Analysis 1 - Overall Evaluation of Fuzzer

This section will explain the results obtained from the system after several runs with increasing the runtime. As mentioned in the Section 5.2, the system was run in a controlled environment to observe the system consistency during the validation process.

The system was run on Ubuntu version 24.04.1 LTS with a logical core 16. Here, we analyze the overall fuzzing consistency of the system. As the system was run several times, each time it produced a different number of iterations for a fixed run time.

Run	Runtime (s)	Iter.	Div.	Div. Rate (%)	iter/s	Fallback	Accepted	Rejected
Result 1	1212	131	11	8.4	0.108	45	58	17
Result 2	1201	151	14	9.3	0.126	53	61	23
Result 3	505.5	62	8	12.9	0.123	17	31	6
Result 4	3600.8	450	44	9.8	0.125	144	199	63
Total	6519.3	794	77	9.7	0.122	259	349	109

Table 5.1: Overall Fuzzing Summary

As shown in the Table 5.1, the first execution produced 131 iterations in 1212 seconds, whereas the third test completed 500 number of iterations in 62 seconds and the fourth run had 450 iterations in 3600 seconds. If we look at the Table 5.1, there is an output that was produced two times for the same timeline. The iteration count for the same runtime varies, as each iteration process depends on how long the validators take to process mutated RRDP files.

If we look at all the test results, the system was consistently identifying the actual divergence between the validators. During the First run, the system produced true divergence 11 times, in 2nd round 14 times for the same timeline. And the number of test cases that generate the divergences across validators increases with the increasing number of iterations. As the system is producing the same type of differentials in every independent run, rather than producing random differentials states that the fuzzer is reliably triggering true behavior. In addition, the divergence rate remains stable across all the executions. this stable consistency across all the runs for different durations shows that the fuzzer reliably triggers divergent behavior at a predictable rate.

Looking at the overall fuzzing summary, the system also identified the *rpki-client* transport fallback and filtered them as raw differential behavior. According to the *rpki-client*, the

validator falls back to rsync or its local cache whenever RRDP fails during the validation process. This type of divergence is excluded from actual divergence as not all the validators are using the same data and the same transport protocol. Whenever rpki-client falls back to rsync, the generated output reflects the cache repository instead of the RRDP mutated repository. This type of transport fallback cases are classified as a raw differential, as classifying them as true divergences would conflict with parsing errors or VRP disagreements. These fallback cases are recorded under confounded differentials as they can reveal implementation differences in how validators handle RRDP failures.

The remaining part of the results shows a result of the total number of test cases that are accepted and rejected by validators.

The overall fuzzing summary from all the experiments shows that the generated results are stable and reproducible. During the period of fuzzing, the Routinator always rejected a malformed RRDP input and produced a zero vrps list, whereas other validators, rpki-client, and FORT generated an output with 100 VRPs list on the same input. This consistent behavior was observed during every run across different inputs.

5.4 Analysis 2 - Inconsistent Behavior Across the Validators

This section analyzes the actual inconsistencies discovered by the system during the validation process, and the XML mutations that cause the validators to disagree. As mentioned in Section 5.2 and Section 5.3, the system was run several times to observe its consistent and repeatable behavior on producing symmetrical differentials.

Out of the total 794 iterations, the divergence was observed for 77 times in total. From these 77 test cases, four selective cases will be analyzed in detail on the mutation that causes the identical differential behavior across validators every time. Each of them defines different types of mutations.

The pattern of log validators showed during the execution:

- **Routinator:** The validator aborts the TLS connection when processing a mutated structurally valid `snapshot.xml`, reporting it as `malformed XML`.

Log: The pattern of log validators showed during the execution:

- **Routinator:** The validator produced 0 VRPs list when processing a mutated structurally valid `snapshot.xml`, reporting it as `malformed XML`.

routinator.log:

```
[WARN] RRDp https://my.server.com/data/rrdp/notification.xml:
Failed to process snapshot file XML
https://my.server.com/data/rrdp/
4b3d782e-9f5b-461a-9963-229203464ad9/1/
24b78e9f0aeb7827/snapshot.xml: malformed XML

[INFO] RRDp https://my.server.com/data/rrdp/notification.xml:
Update failed, and there is no current copy.
```

- **rpki-client:** For the same test case rpki-client successfully fetched the full snapshot file and produced output with list 100 VRPS.

rpki-client log: Log:

```
rpki-client: ta/ta: pulling from https://my.server.com/
data/repo/ta/ta.cer
rpki-client: ta/ta: loaded from network
rpki-client: https://my.server.com/data/rrdp/notification.xml: pulling from network
rpki-client: https://my.server.com/data/rrdp/notification.xml: downloading snapshot (78788f77-c974-4b2c-b677-dddb27c5aafe#1)
rpki-client: https://my.server.com/data/rrdp/notification.xml: loaded from network
rpki-client: all files parsed: generating output
[rpki-client: 100 VRPs generated]
```

- **FORT:** For the same input, FORT has also parsed the file and generated an output of 100 Vrms list. FORT reports only the final validation result rather than a detailed processing log upon successful validation.

Log:

```
[fort: 100 VRPs generated]
```

Different types of disagreement were observed across the validators during execution. We have classified test cases by comparing their vrps lists.

Pattern (R / C / F)	Occurrences	Status	Meaning	Interpretation
0 / 100 / 100	77	Clean	Rejected by Routinator	Remained stable across replay and passed metadata consistency checks. The representative case (<code>iter_00014</code>) was replayed multiple times, preserved the same <code>session_id</code> , snapshot URI, and snapshot hash, and kept producing the same pattern.
0 / 0 / 100	39 (structural mutations)	Raw	Routinator and rpki-client reject	A raw disagreement pattern, not automatically a clean differential. May arise from repository inconsistency or other Oracle-rejected conditions.
100 / 0 / 100	220 (serial mismatch, no valid manifest, bad message digest)	Raw	Rejected by rpki-client	These raw patterns presented as raw differential logs before filtering. Representative examples show rpki-client rejecting with no valid manifest.
0 / 0 / 0	109	Not a differential	All reject	Universal rejection, not validator inconsistency. All validators agree and reject.

Table 5.2: Classification of VRP output patterns across validators. R = Routinator, C = rpki-client, F = Fort. Values indicate VRP count (0 = rejected, 100 = accepted). Status indicates whether the pattern qualifies as a clean differential after replay and metadata consistency checks.

As shown in the following Table 5.2, there are 77 test cases that got rejected by Routinator and produced 0 VRPs lists that followed the same pattern. This one classified pattern shows that the divergence originates from Routinator’s stricter XML parsing behavior rather than an implementation error. Among all patterns shown in Table 5.2, we will discuss Rotinator’s rejection class, as it produced the same parser issue in every test case replay.

5.4.1 Understanding of Parser Differential Behavior for Routinator

During the validation process, the differential Oracle has identified several divergences across the validators. One notable test case is iter_00014, which contains a well-formed XML snapshot.xml file referenced in the notification.xml file. These snapshot files contain an XML Processing Instruction (PI). Whenever Routinator parses these XML files, it rejects the snapshot file, stating malformed XML, and produces 0 VRPs, whereas rpki-client and FORT both kept parsing the same repository and produced an output of 100 VRPs. Since the hash values of the snapshot.xml file remain consistent with the hash value advertised in the actual notification.xml file, the disagreement was not a random logic error or inconsistency.

To understand the behavior, we looked at the XML parsing path of Routinator. This rejection points to the Routinator’s rpki-rs library, a file name src/xml/decode.rs. When looking at this file, we found that a function called Content::take_element(...) accepts only a limited set of XML event types; otherwise, it returns malformed. Table 5.3 summarizes how the rpki-rs library handles each XML event type.

XML Event	rpki-rs handling	Well-formed?	In RELAX NG?
Event::Start	Processed	Yes	Yes
Event::End	Processed	Yes	Yes
Event::Text	Processed	Yes	Yes
Event::Comment	Not explicitly handled	Yes	No
Event::Decl	Not explicitly handled	Yes	No
Event::DocType	Not explicitly handled	Yes	No
Event::PI	Error::treated as Malformed	Yes	No

Table 5.3: XML event handling in rpki-rs and its relationship to well-formedness and the RELAX NG schema.

If we look at the PI event shown in Table 5.3, the PI injected a well-formed XML file that caused Routinator to reject a valid XML file as malformed XML, where other XML events, such as comments, document, and declarations, get skipped silently.

5.4.2 RFC 8182 XML Handling

The previous section showed that Routinator rejects snapshot files containing a PI. This raises the question of whether such a rejection is required by the specification. According to RFC 8182 (Section 3.5.2.3), validators must reject any snapshot file that is not well-formed or does not follow the RELAX NG schema described in Section 3.5.4 [3]. However, a snapshot XML file containing a Processing Instruction remains well-formed as per the XML 1.0 specification, and PIs can be included anywhere in the XML document [30]. Additionally, the RELAX NG schema specified in RFC 8182 only talks about the XML elements and attributes, but there is no mention of the Processing Instructions [31]. To verify this, we validated the snapshot file with the PI using the `xmllint` command, and the result was successful in both cases, which means that the file was both well-formed and valid according to the RELAX NG schema.

This outcome is important as it exemplifies that the differential fuzzer can uncover parser-level implementation differences across validators. In other words, the finding is not limited to only repository-level inconsistencies. Even if the repository is valid, well-formed, and reproducible, validators can still disagree because Routinator, rpki-client, and FORT handle XML differently.

5.4.3 Analysis of Mutated XML Files Causing Disagreement

After identifying the mutated XML seeds, we analyzed the kinds of XML mutations associated with this disagreement. The analysis looks only at the mutated XML files. It does not examine how validators behave on the mutated data. The mutated files can contain different types of modifications, which means that some modifications can overlap.

Based on the study in [1], testing of the structure-aware differential fuzzer for the RPKI validation pipeline has been conducted, but it does not explicitly examine the XML files in RRDP. To examine XML-level structural mutations, a low-level mutation strategy was used to determine whether such mutations may affect validators. As mentioned in Section 4.2.2, XML-level modifications during the validation process can take the form of comment injection, CDATA, and name spacing. In particular, CDATA appeared in 247 iterations,

unknown attributes appeared in 236 iterations, and processing instructions appeared in 210 iterations. Unknown custom elements appeared in 0 tests. These results confirm that the mutator often introduces changes to the XML that are relevant to the parser. However, this does not mean that these mutations necessarily caused validation failures. Rather, the mutator modifies the XML in ways that may affect the validator's behavior.

Construct	Iterations
Processing instructions	210
CDATA sections	247
Unknown custom attributes	236
Unknown custom elements	0

Table 5.4: Appearance of Targeted Mutation corpus.

5.4.4 Unknown Attributes

Following the previous Section 5.4.3, we analyzed which type of attribute names actually appeared during the XML mutations. There are some categories of names that appeared repeatedly in the XML files. The types of names were x-priority, x-ttl, x-encoding, x-type, x-format, and x-source. Each of the names was repeated for 229 times. The other unknown attributes, like x-fuzz-* and things like that, these types of unique attributes were found in the form of 93 category and they showed up for 302 times in 19 iterations. This concludes that these unknown attribute mutations were not random noise from the fuzzer. The system was able to find one repeated recurring family, plus a small exploratory fuzz tail.

Attribute group	Hits	Iterations
x-priority	2071	229
x-ttl	2021	229
x-encoding	2011	229
x-type	1983	229
x-format	1978	229
x-source	1927	229
x-fuzz-* tail	302	19

Table 5.5: Types and repetition of unknown attribute that appeared in xml files

5.4.5 Examples of unknown Attribute Mutations in XML Files

After all these analyses, there are 939 test cases in total that have been stored across the runs. Some of the test cases have been presented in the following table to show which mutations caused the Routinator to disagree with other validators.

Iter	Case	Document	Mutation	Position	Interpretation
iter_00035	11fbfd37 ...	snapshot. xml	Repeated semantic unknown attributes: x-ttl, x-format, x-encoding, x-priority, x-source, x-type	Inside publish elements	Dominant recurring semantic custom-attribute family
iter_00048	c8a8e69a ...	snapshot. xml	Repeated wrapper-depth attributes: depth="3", depth="2", depth="1"	On container, bundle and group	Recurring structural wrapper-attribute family
iter_00013	419e13ab ... + c8a8e69a ...	snapshot. xml	Compound unknown-attribute case: root-level x-fuzz-*, publish-level x-fuzz-*, plus semantic custom attributes	On snapshot root and publish elements	Rare compound case combining multiple unknown-attribute styles

Table 5.6: Representative unknown-attribute mutation cases.

5.4.6 False Positive Analysis

Based on the previous work on the differential fuzzer [1], [32] [3] [14], it is possible that the system can generate false positive results. This can happen when the generated test results are inconsistent across repeated runs, or the test environment is not properly controlled.

To verify this differential oracle needs to check the validity of the XML structure, rpki-client, rsync and cache fallback behaviour. These helps oracle to filter out the real test cases. In RRDP, if any mutations cause the validators to behave differently, the repository metadata should be checked. For example, if the advertised hash on the **Snapshot.xml** file does not match the hash referenced in **notification.xml** file, the observed disagreement behavior in

validators is caused by the inconsistent RRDp base64 data rather than an XML mutation. This type of result comes from a hash mismatch, not from mutations.

To reduce the risk of false positive results, the system verifies the hash consistency of the notification and snapshot file before considering it as a real divergence.

Field	Value
Case directory	data/oracle/cases/iter_00014
Notification file	data/oracle/cases/iter_00014/ notification.xml
Snapshot URI	https://my.server.com/data/rrdp/fee7734f-934d-4e0b- b504- 93f8f1cd4e1d/1/40673705b1a5dfc7/ snapshot.xml
Local snapshot file	data/oracle/cases/iter_00014/fee7734f-934d-4e0b- b504- 93f8f1cd4e1d/1/40673705b1a5dfc7/ snapshot.xml
Pre-mutation hash	ca85b90037998826a089fff424db77b56f881604 ff959bd6974b62496ebdcaa1
Post-mutation file hash	ca85b90037998826a089fff424db77b56f881604 ff959bd6974b62496ebdcaa1
Result	Match

Table 5.7: Verification of snapshot hash consistency for the representative case iter_00014, this case shows inconsistencies across validator.

5.4.7 Test cases Repeatability

To check the differential Fuzzer stability, the test cases stored by the differential oracle were reproduced manually under the same configuration and environment. This step is essential for identifying false-positive results caused by environmental noise rather than

RRDP XML mutations. For this purpose, the test case number `iter_00014` was run manually for 6 times to check whether it produced the same results. This shows that the differential fuzzer consistently reproduced the same behavioral differences. The example `iter_00014` repeated output is shown below:

Run	Case	Routinator VRPs	rpki-client VRPs	Fort VRPs
1	<code>iter_00014</code>	0	100	100
2	<code>iter_00014</code>	0	100	100
3	<code>iter_00014</code>	0	100	100
4	<code>iter_00014</code>	0	100	100
5	<code>iter_00014</code>	0	100	100
6	<code>iter_00014</code>	0	100	100

Table 5.8: Repeatability of replay for oracle case `iter_00014`

The Table 5.8 shows the outcome of reproducibility. The case number `iter_00014` was run for 6 times manually using the same repository state. Before each run, the validator caches were cleared and every time the stored repository state was rerun. The case that Oracle stored was verified with the deployed `notification.xml` file, including the session identifier, snapshot URI and snapshot hash.

If we look at the result for 6 times replay, the **Routinator** produced 0 VRPs every time. On the other side, **rpki-client** and **Fort** produced 100 VRPs consistently. This means that the disagreement is not a one-time thing. It is stable. Can be repeated.

This repeatable run across the validators confirms that the differential fuzzer is stable and it consistently generating same output for the same RRDP mutated input.

The reproducibility test above was performed on the mutated test case `iter_00014`, to make sure that the mutation diversity identified in Table 5.6 reflects genuinely distinct structural changes. Inspection was made on the stored `snapshot.xml` files for the other representative scenarios. The `iter_00035` case contains semantic unknown attributes (`x-ttl`, `x-format`, `x-encoding`, `x-priority`, `x-source`, `x-type`), `iter_00048` contains wrapper-depth attributes (`depth="1"`, `depth="2"`, `depth="3"`), and `iter_00013` contains a compound pattern combining `x-fuzz-*` attributes with semantic custom attributes. All four scenarios represent structurally distinct mutation families that confirm the differential oracle captures divergences triggered by different mutation strategies, not a single repeated pattern.



Replay-based reproducibility verification for these additional cases is left as part of future validation efforts.

Test case	Field	Live deployment	Oracle case
iter_00014	Session ID	fee7734f-934d-4e0b-b504-93f8f1cd4e1d	fee7734f-934d-4e0b-b504-93f8f1cd4e1d
	Snapshot URI	https://my.server.com/data/rrdp/fee7734f-934d-4e0b-b504-93f8f1cd4e1d/1/40673705b1a5dfc7/snapshot.xml	https://my.server.com/data/rrdp/fee7734f-934d-4e0b-b504-93f8f1cd4e1d/1/40673705b1a5dfc7/snapshot.xml
	Snapshot hash	ca85b90037998826a089fff424db77b56f881604ff959bd6974b62496ebdcaa1	ca85b90037998826a089fff424db77b56f881604ff959bd6974b62496ebdcaa1

Table 5.9: Verification of replay metadata consistency for oracle case iter_00014.

The Table 5.9 shows that the iter_00014 has the same session ID, snapshot URI, and snapshot hash for live-served and stored oracle test case.

5.5 Analysis 3 - Raw Differentials

This section describes the compound or raw differentials before Oracle filtering. This analysis is mainly focused on how the system identifies them as raw differentials by comparing their metadata inconsistency, valid XML structure, rather than clean RRDP-attributable validator differences.

5.5.1 Overview of Raw Differentials

The disagreement test cases that were discovered during the fuzzing are considered as raw differentials. This happens before the Oracle filtration. Especially, these type cases occur

when the validators generate different outputs for the same mutated repository. This kind of disagreement does not automatically specify that there are real divergences or that the validators are behaving differently. These disagreements appear whenever the validators see a vector like mismatched RRDp base64 metadata, broken repository setup, a validator falling back to rsync or its local cache, or the snapshot not being fully retrieved.

To check that, the Oracle does additional validations before storing the confirmed test cases. In this filtration process, if the generated output of validators differs after the mutations, the Oracle excludes these test cases. As discussed in earlier Section 4.4.4, if some of the test cases show real differential behavior, other cases show inconsistent behavior as the mutated repository itself is inconsistent. For example, if the Oracle finds the metadata, session_id and snapshot hash of the **notification.xml** file do not match the metadata, session_id and hash value advertised in the actual **notification.xml**, then it discards all these test cases.

The Oracle not only checks the metadata or hash value, it also verifies whether the validator **rpki-client** fell back to rsync or its cache. As this transport falls back can also generate validator disagreement, so the Oracle excludes such cases to ensure that only the mutated input is identified as the cause of a divergence.

To check the stability and repeatability of validator disagreements, the oracle runs replay executions on the test cases. Only in the case when the repository metadata matches what was stored and the validator outputs remain consistent over several runs is a case retained as a confirmed differential. If not, the cases are discarded from the overall results and counted as raw or confounded differential.

Basically, the differential Oracle confirms only the genuine validators disagreement by filtering out repository problems or fallback behavior.

5.5.2 Raw Differential Logs for a Structural RRDp Mutation

The following logs present a compound differential division for a structural mutation of the RRDp XML files. The example is shown at the outcome level to illustrate how the validators reacted differently to the same mutated repository state.

- **Routinator:** For this representative raw differential case, Routinator accepted the mutated repository state and generated 100 VRPs.

Log:

```
routinator -> ACCEPTED (100 VRPs)
```

- **rpki-client:** For the same test case, **rpki-client** rejected the repository during validation, reporting that no valid manifest was available and therefore producing 0 VRPs.

Log:

```
rpki-client: my.server.com/data/repo/ta/
             d3ec349f6ae5f69d74d6e941d48420a171fef6be.mft:
no valid manifest available
rpki-client: all files parsed: generating output
[rpki-client: 0 VRPs generated]
```

- **FORT:** For the same mutated input, FORT accepted the repository state and generated 100 VRPs.

Log:

```
[fort verbose output:]
[fort: 100 VRPs generated]
```

The three logs that show a raw differential for the same structurally mutated RRDP case: Routinator and FORT accepted the mutated repository state and generated 100 VRPs, whereas **rpki-client** rejected the case during validation and produced 0 VRPs stating no valid manifest.

5.5.3 Analysis of Dataset

Here, this section describes the analysis of oracle cases that are stored in the `data/oracle/-cases/iter_*` folder. Each case contains a mutated **notification.xml** file and some other output files, such as `mut.norm`, `added.txt`, and `removed.txt`, which are generated by validators.

IF we look at the Table 5.10, for every iteration, the number of outputs is different.

In this section, there are three different types of counting units. An iteration refers to the execution of the fuzzing loop. An archived case refers to the Oracle case stored in `data/oracle/cases/iter_*`, which includes the mutated repository metadata as well as the Oracle output, if it has been saved. A mutated snapshot instance refers to an occurrence of the analyzed `snapshot.xml` file that has been collected for structural mutation analysis, and the number of mutated snapshot instances exceeds the number of archived cases because there can be more than one mutated instances per archived case.

Validator / Scope	Archived iterations	Artifacts available	Used in joint 3-validator comparison
Routinator	757	<code>mut.norm</code> , <code>added.txt</code> , <code>removed.txt</code>	Yes
<code>rpki-client</code>	898	<code>mut.norm</code> , <code>added.txt</code> , <code>removed.txt</code>	Yes
Fort	181	<code>mut.norm</code> , <code>added.txt</code> , <code>removed.txt</code> , where present	Yes
Common case-level file	939	<code>Mutated notification.xml</code>	Applies to all archived cases
Common output (Routinator, <code>rpki-client</code> , Fort)	181	Common archived validator outputs	Comparison subset

Table 5.10: Archived artifact coverage by validator and size of the common three validators comparison subset.

A pattern here is defined by the combination of mutation categories found in one analyzed `snapshot.xml` instance, not in a single case by itself.

Uniqueness of Structural Mutation Distribution in Snapshot.xml: To understand the structural mutation diversity that drives differential behavior across RRDP XML files, we analyzed mutation patterns across all mutated snapshot files (939) that caused divergences. We categorized each snapshot.xml file by its mutation type(s), then grouped snapshots.xml files with the same pattern into families. After analyzing them, we found the 1576 unique patterns of mutations, which show that the fuzzer can produce a variety of mutations across the XML files.

Type of Mutations: From the following Table 5.11, there are 10 types of mutations found across all the snapshot files. Child reordering is the most common mutation occurred in every step of the iterations by changing the order of <publish> elements inside a <snapshot>. As mentioned in the RELAX NG schema in RFC 8182 (Section 3.5.4) does not require any specific ordering of <publish> elements, so the reordered snapshots are valid according to the specification. Despite permutations, the other nine mutation types appear at varying rates. For example, one snapshot.xml file can contain more than one mutation pattern at the same time.

Mutation Type	Snapshots	% of Total
Child reordering (publish elements)	4296	100.0%
Unknown elements	400	9.3%
Duplicate publish URIs	373	8.7%
Processing instructions	324	7.5%
Depth attributes	313	7.3%
Wrapper elements	313	7.3%
Unknown attributes	303	7.1%
CDATA sections	298	6.9%
Namespace prefixes	206	4.8%
Comments	127	3.0%
<i>Coarse family analysis</i>		
Total mutated snapshots	4296	
Distinct structural patterns	1576	
Singleton families (unique)	1418	90.0%

Table 5.11: Distribution of mutation patterns identified across all mutated snapshot.xml files, grouped by coarse family analysis. Percentages are relative to the total number of mutated snapshots (4296).

From this analysis, we can see that the mutated snapshot.xml file contains a wide range of mutation instances. But the mutations are not very different from each other in terms of structure. There are repeated instances of similar mutation families, and these instances are very different and concrete.

5.6 Contextualization of Results

To put the results into context, we have compared the results with the results obtained in the paper on CURE [1], which fuzzes the entire RPKI Relying Party validation process with standardized RPKI objects such as certificates, manifests, CRLs, and ROAs. The tool generated over 600 million test cases and found 18 vulnerabilities with direct security implications. In this thesis, we consider the RRDP XML transport and parsing path, with 794 iterations. The objective is different. Instead of searching for validation flaws or crash vulnerabilities with objects, we explore whether there is disagreement between independent validators on accepting and parsing unexpected mutated input when the XML files are well-formed. The disagreement between validators indicates ambiguity in the specification and differences in the implementation of the RRDP XML processing layer. Furthermore, this framework was always able to find differences that happened over and over again, in every execution.

This shows that the RRDP-Diff fuzz system is effective and robust at finding behavioral inconsistencies among RPKI validators.

6 Limitations

The approach of Differential Fuzzer using XML mutation strategy allows systematic exploration of RRDP validators' behavior. Despite the promising output, it comes with several inherent limitations.

- **Mutations Do Not Guarantee to Preserve RRDP State-Transition Semantics:** After mutating the RRDP XML files by applying the mutation engine, it does not guarantee that the mutated repository remains semantically valid even after generating well-formed XML files. For example, a notification file containing a `session_id`, serial number, hash, or referenced object that does not match the expected RRDP repository state. This can lead to a validator disagreement, and the differential behavior may stem from repository inconsistencies rather than from the implementations.
 - **Reason:** This limitation comes from the methodology used for mutations using the Structure-Aware XML Mutation technique. The main purpose of this method is to keep the XML structure valid so that it can pass through deeper RRDP validation logic without failing at the front door during the validation process. As this mutation approach preserves the structure of XML well-formedness, it mutates only the protocol fields such as `session_id`, serial numbers, hashes, URIs, and structural elements. As a result, it generates syntactically valid XML files that may violate RRDP consistency requirements across files.
 - **Complexity:** According to the RFC8182 RRDP, semantic correctness can not be defined by one XML file as it depends on the relationship between notification, snapshot, and delta files [3]. This includes session continuity, serial progression, and hash consistency, and enforces all these constraints after each mutation to maintain the state of repository evolution. This increases the complexity of the system. The earlier work CURE also identifies this specification complexity as a core challenge in testing RPKI validators [1].

-
- **Bounded Mutation Coverage on Delta Path Update:** In this experiment, the system mainly applies mutations to the snapshot.xml and notification.xml files. As the delta.xml files are missing in this system, the update increment path is not tested. Because of this, the system does not completely check the RRDP synchronization update behavior.
 - **Reason:** This issue comes from the repository generator. The current repository contains only the snapshot and the notification file; due to this, the current system is designed to mutate only on the available file.
 - **Complexity:** To support delta-path testing, the system would need to generate and maintain a consistent repository history across multiple steps. This is because RFC 8182 states that delta elements must be in sequence, and validators must process them in order rather than separately.
 - **Partial Stateful Behavior:** In this approach, the fuzzer does not depend on previous iterations. Since RRDP works as a stateful protocol with continuous sessions in the real world, some state-dependent violations may be overlooked.
 - **Reason:** In this test workflow, the fuzzer starts with a fresh copy of the repository after each run. Then, it applies mutations in a loop. This framework focuses only on individual repositories instead of tracking session IDs and serial numbers across iterations.
 - **Complexity:** Based on earlier research, we know that tracking previous sessions is one of the hardest parts of RPKI fuzzing [1]. In RRDP, it is necessary to check that the changes are made in the correct order over time, so working on this would require a much more complex system that can keep track of all these things.
 - **Untested Extended RRDP Standards:** The current system does not test the same-origin rule of RRDP from RFC 9674 [18] and RFC 9697 [33]. For example, validators must verify that the URLs of the snapshot and delta file come from the same server as the notification file. The mentioned RFCs also added a recovery rule that the framework does not test. In the recovery rule, the validators must identify if a hash of a delta for the same session_id and serial number changes between successive Update Notification Files.

-
- **Reason:** In the current setup, the mutation workflow focuses on the XML level mutations. Although the current mutation crate modifies URIs and hash value within a single repository, it does not support the cross-origin cases or multiple successive update cycles.
 - **Complexity:** To test these rules, the system needs a more complex setup. For RFC 9674, the system requires control over multiple origins at the same time. For RFC 9697, the system needs to keep track of multiple successive updates in order, using the same validator each time.
 - **Limited Root-Cause Localization in Black-Box Mode:** The framework uses differential Oracle to detect divergence. The system can only identify the inconsistencies, but it can not precisely identify which part of the validator caused it.
 - **Reason:** Right now, the system only observes external outputs such as logs, exit codes, runtime data, and VRP sets. However, it does not see the validators' internal parser states, call traces, or memory state. This design decision maintains the framework's portability and enables validators to be tested in black-box mode.
 - **Complexity:** Since all the validators are designed and implemented individually, adding a layer for tracing, coverage code, and debug modes would increase complexity [1].
 - **False Positive:** Even though the current system filters for true divergence and raw differentials, the system can still produce false positive results. The result stored by the system requires a manual test to analyze the differentials. To eliminate the false positive results, we need to know the correct validator behavior for every possible input. The paper [4] says that not having a clear way to define what is correct is a big problem when testing RPKI validators.
 - The implemented validators in this system are running on hard-coded paths and this limits its portability across different environments.
 - A strict XML 1.0 parser is more restrictive than the Xmltree Rust library used in the AST manipulation. However, some XML-invalidating mutations managed to parse correctly in the mutator. When feasible, the external xmllint tool is used by the triage system as a stringent XML well-formedness check.

-
- Injection must be restricted within element boundaries to use the sentinel-based solution for XML comment, CDATA section, and processing instruction injections (xmltree cannot have these node types). Textual content and attribute values cannot contain these constructs [8], [19], [20]

6.1 Future Improvement

This section discusses the improvements of the existing framework to make the system more effective and easier to use.

- **Timeout Handling:** Timeout handling of the current system is not separated from actual validation failures. This can lead to false positive results. Improving timeout events would help to increase the system's reliability and accuracy of the output.
- **RP Implementation Coverage:** "The framework currently supports three RP implementations: Routinator, Fort-validator, and RPKI-Client. Adding support for OctoRPKI would improve implementation coverage and increase the chances of detecting implementation-related differences.
- Following the confined validators hard-coding implementation, an improvement could be extended to run each validator inside a Docker container. Run them by invoking directly from Rust. This would remove the hard coded paths and make the system portable across different environments.
- **Mutation Diversity:** The `xml_mutator` can be improved by extending its additional mutation operators to generate a wider variety of test cases.

6.2 Future Work

This section introduces several interesting directions for future research.

- **Delta Fuzzing:** Extend this system to be able to mutate and serve delta-chain updates based on session and serial states for RRDP. As RFC 8182 also states, delta processing is a part of RRDP synchronization and would strengthen the research coverage of this work.
- **Multi-Round Stateful Fuzzing:** Another big contribution would be to make the fuzzer keep track of its state. This means keeping a record of `session_id`, serial numbers, and how things change over time across runs. This future work will help the fuzzer find bugs that depend on the RRDP state and are hidden in the current design, which starts from a fresh repository state each time. Making the fuzzer stateful helps to test session continuity and serial number-based behavior according to RFC 8182 [3] rules.
- **Systematic Testing of RRDP Protocol Standards:** A further extension of this work would be to include systematic tests for RRDP updates. These RRDP updates introduce a same-origin policy rule that requires validators to verify that snapshot and delta URIs are from the same origin as the notification URI. Another RRDP update adds detection and recovery logic for delta-hash desynchronization across update notifications. Implementing test cases for these RRDP standards paths would help ensure that validators implement them correctly.
- **Automated Root-Cause Analysis:** Another future approach would be adding an automated root-cause analysis. This implementation is complex, as the validators are designed individually. However, such an extension would reduce the manual work required to investigate each detected divergence.
- **Implementation of Smart Oracle:** Developing a smart oracle would help classify detected divergences more clearly. Basically, it would help the fuzzer distinguish parser-level failures, metadata inconsistencies, transport fallback cases, and true validator behavioral differences.

7 Conclusion

This research presents the RRDP-Diffuzz framework, a differential fuzzing system for testing RRDP protocol-based RPKI validators in a controlled environment. By implementing structure-aware XML mutation, repeated validator execution, and a differential oracle together, we were able to build a framework that can detect behavioral disagreement or inconsistencies across multiple validators.

The evaluation of the RRDP-Diffuzz framework showed that it can generate various types of mutated RRDP repositories, and by using these repositories, the system identified valuable differential behavior across validators. Every test case that was marked as a cause of validator divergence was reproducible, and all of them were well-formed snapshot.xml files. The snapshot.xml files that contained a Processing Instruction (PI) were rejected by Routinator, while rpki-client and FORT continued processing the same repository state. This result shows that the system is not only limited to repository-level inconsistencies, but can also uncover parser-level implementation differences.

Furthermore, the overall analysis of all the outputs illustrates that differential fuzzing is an effective and viable approach for systematically uncovering behavioral inconsistencies among RRDP validators. Even though the current framework acts as a black box, the system lacks full delta-chain support and complete protocol coverage; these limitations reflect a deliberate first step toward deeper investigation in future work.

8 Acknowledgment

I would like to express my sincere gratitude to my supervisors for their guidance, support, and encouragement throughout the course of this research. Their expertise and thoughtful insights have been invaluable at every stage of this work, and their patience and constructive feedback kept me motivated throughout the process.

This work would not have been possible without each of you. Special thanks are due to Prof. Dr. Haya Schulmann for giving me this opportunity, and to Prof. Dr. Michael Waidner for his supervision and guidance throughout this work. I would also like to thank M.Sc. Niklas Vogel for his continuous support, valuable discussions, and helpful feedback during the development of this research. I am also grateful to Prof. Dr. Björn Scheuermann for his support and co-supervision in this thesis process.

I am deeply thankful to my family for their unconditional love and support. Their encouragement has been my greatest motivation and has helped me stay focused throughout this journey. To my friends, thank you for your kindness and for always believing in me. Your support gave me the strength to keep going, and the time spent together made this academic journey truly special.

Bibliography

- [1] D. Mirdita, H. Schulmann, N. Vogel, and M. Waidner, “The CURE to Vulnerabilities in RPKI Validation,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, Internet Society, 2024. doi: 10.14722/ndss.2024.241093. [Online]. Available: <https://www.ndss-symposium.org/ndss-paper/the-cure-to-vulnerabilities-in-rpki-validation/>.
- [2] D. Goodin, *Suspicious event hijacks Amazon traffic for 2 hours, steals cryptocurrency*, Ars Technica, Apr. 2018. [Online]. Available: <https://arstechnica.com/information-technology/2018/04/suspicious-event-hijacks-amazon-traffic-for-2-hours-stealing-cryptocurrency/>.
- [3] T. Bruijnzeels, O. Muravskiy, B. Weber, and R. Austein, *The RPKI Repository Delta Protocol (RRDP)*, RFC 8182, Jul. 2017. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8182>.
- [4] H. Schulmann, N. Vogel, and M. Waidner, “RPKI: Not Perfect But Good Enough,” 2024. arXiv: 2409.14518 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/2409.14518>.
- [5] D. Mirdita, H. Schulmann, and M. Waidner, “SoK: An introspective analysis of RPKI security,” in *34th USENIX Security Symposium*, USENIX Association, 2025. [Online]. Available: <https://www.usenix.org/system/files/usenixsecurity25-mirdita.pdf>.
- [6] T. Hlavacek, P. Jeitner, D. Mirdita, H. Schulmann, and M. Waidner, “Beyond limits: How to disable validators in secure networks,” in *Proceedings of the ACM SIGCOMM 2023 Conference*, ACM, 2023, pp. 950–966. doi: 10.1145/3603269.3604822.
- [7] X. Zhang et al., “A survey of protocol fuzzing,” *ACM Computing Surveys*, vol. 57, no. 2, pp. 1–36, 2025. doi: 10.1145/3696788.
- [8] OWASP Foundation, *WSTG - Input Validation Testing (WSTG-INPV-07)*, Accessed: March 2026, 2024. [Online]. Available: https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/.

-
-
- [9] J. Somorovsky, A. Mayer, J. Schwenk, M. Kampmann, and M. Jensen, “On Breaking SAML: Be Whoever You Want to Be,” in *21st USENIX Security Symposium*, USENIX Association, 2012. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity12/technical-sessions/presentation/somorovsky>.
 - [10] S. Nilizadeh, Y. Noller, and C. S. Păsăreanu, “DifFuzz: Differential Fuzzing for Side-Channel Analysis,” in *IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, IEEE, 2019, pp. 176–187. doi: 10.1109/ICSE.2019.00034.
 - [11] V.-T. Pham, M. Böhme, and A. Roychoudhury, “AFLNET: A Greybox Fuzzer for Network Protocols,” in *IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*, IEEE, 2020, pp. 460–465. doi: 10.1109/ICST46399.2020.00062.
 - [12] R. Natella, “StateAFL: Greybox Fuzzing for Stateful Network Servers,” in *Empirical Software Engineering*, vol. 27, Springer, 2022. doi: 10.1007/s10664-022-10233-3.
 - [13] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse, “AFL++: Combining incremental steps of fuzzing research,” in *14th USENIX Workshop on Offensive Technologies (WOOT)*, USENIX Association, 2020. [Online]. Available: <https://www.usenix.org/conference/woot20/presentation/fioraldi>.
 - [14] S. Srinivasan, T. Menzies, and M. D’Amorim, “SmartOracle – An Agentic Approach to Mitigate Noise in Differential Oracles,” 2025. doi: 10.48550/arXiv.2601.15074. arXiv: 2601.15074 [cs.SE]. [Online]. Available: <https://arxiv.org/abs/2601.15074>.
 - [15] N. Havrikov and A. Zeller, “Systematically Covering Input Structure,” in *IEEE/ACM 34th International Conference on Automated Software Engineering (ASE)*, IEEE, 2019. doi: 10.1109/ASE.2019.00027.
 - [16] J. Wang, B. Chen, L. Wei, and Y. Liu, “Superion: Grammar-Aware Greybox Fuzzing,” in *IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, IEEE, 2019. doi: 10.1109/ICSE.2019.00081.
 - [17] Bright Security, *Understanding XML Injection: Risks, Prevention, and Best Practices*, Accessed: March 2026, 2024. [Online]. Available: <https://brightsec.com/blog/understanding-xml-injection-risks-prevention-and-best-practices/>.
 - [18] J. Snijders, *Same-origin policy for the RPKI repository delta protocol (RRDP)*, RFC 9674, Updates RFC 8182, Dec. 2024. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9674>.

-
-
- [19] OWASP Foundation, *XML Security Cheat Sheet*, Accessed: March 2026, 2024. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/XML_Security_Cheat_Sheet.html.
- [20] OWASP Foundation, *XML External Entity (XXE) Prevention*, Accessed: March 2026, 2024. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/XML_External_Entity_Prevention_Cheat_Sheet.html.
- [21] Y. Rekhter, T. Li, and S. Hares, *A border gateway protocol 4 (BGP-4)*, RFC 4271, Jan. 2006. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc4271>.
- [22] H. Ballani, P. Francis, and X. Zhang, “A Study of Prefix Hijacking and Interception in the Internet,” in *Proceedings of the ACM SIGCOMM Conference*, Kyoto, Japan: ACM, Aug. 2007, pp. 265–276. doi: 10.1145/1282380.1282411.
- [23] M. Lepinski and S. Kent, *An infrastructure to support secure internet routing*, RFC 6480, 2012. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6480>.
- [24] RIPE NCC Learning and Development, *BGP security webinars: Deploying RPKI*, Webinar slides, Accessed: March 2026, 2025. [Online]. Available: <https://www.ripe.net/training/in-person-training-courses/material/webinar-slides/>.
- [25] R. Bush and R. Austein, *The Resource Public Key Infrastructure (RPKI) to Router Protocol*, RFC 6810, 2013. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6810>.
- [26] R. Bush and R. Austein, *The Resource Public Key Infrastructure (RPKI) to Router Protocol, Version 1*, RFC 8210, 2017. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8210>.
- [27] T. Hlavacek, P. Jeitner, D. Mirdita, H. Shulman, and M. Waidner, “Stalloris: RPKI Downgrade Attack,” in *31st USENIX Security Symposium (USENIX Security 22)*, Boston, MA: USENIX Association, 2022. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/%20presentation/hlavacek>.
- [28] T. Hlavacek, P. Jeitner, D. Mirdita, H. Shulman, and M. Waidner, “Behind the scenes of RPKI,” in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, ACM, 2022, pp. 1413–1426. doi: 10.1145/3548606.3560645.
- [29] C. Aschermann, P. Jauernig, T. Frassetto, A.-R. Sadeghi, T. Holz, and D. Teuchert, “NAUTILUS: Fishing for deep bugs with grammars,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, Internet Society, 2019. doi: 10.14722/ndss.2019.23412.

-
-
- [30] T. Bray, J. Paoli, C. M. Sperberg-McQueen, E. Maler, and F. Yergeau, *Extensible markup language (XML) 1.0 (fifth edition)*, 2008. [Online]. Available: <https://www.w3.org/TR/xml/>.
- [31] J. Clark and M. Murata, *RELAX NG specification*, 2001. [Online]. Available: <https://relaxng.org/spec-20011203.html>.
- [32] L. Wachter, J. Gremminger, C. Wressnegger, M. Payer, and F. Toffalini, “DUMPLING: Fine-grained Differential JavaScript Engine Fuzzing,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, The Internet Society, 2025. doi: 10.14722/ndss.2025.241411. [Online]. Available: <https://www.ndss-symposium.org/ndss-paper/dumpling-fine-grained-differential-javascript-engine-fuzzing/>.
- [33] J. Snijders and T. de Kock, *Detecting RPKI Repository Delta Protocol (RRDP) Session Desynchronization*, RFC 9697, Dec. 2024. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9697>.